



TrackR

DOSSIER PROJET

Portail client white-label pour freelances

Application de gestion de mission : devis, facturation, kanban et espace client en marque blanche.

CANDIDAT

Hamza NAYA

CERTIFICATION

CDA / RNCP37873

PROMOTION

2025 / 2026

La Plateforme, Marseille

Table des matieres

01	Presentation personnelle	3
02	Presentation du projet	4
03	Organisation du projet	8
04	Conception	14
05	Developpement backend	28
06	Developpement frontend	34
07	Tests et qualite	38
08	Deploiement et CI/CD	43
09	Veille technologique	47
10	Bilan et perspectives	50
--	Glossaire	55



01 / PRESENTATION PERSONNELLE

Qui suis-je ?

Je m'appelle Hamza NAYA. A la base j'ai un parcours scientifique : un DUT Mesures Physiques, puis une licence M2E (Materiaux pour l'Energie et l'Electronique). Mais en cours de route j'ai realise que ca ne me correspondait plus vraiment, et que les perspectives apres un master dans ce domaine ne m'attiraient pas.

Ce qui a tout change, c'est un groupe d'amis que j'ai depuis le lycee, tous dans l'informatique. A force d'etre dans leur entourage, j'ai commence a m'y interesser serieusement, et j'ai fini par comprendre que c'est ce que je voulais vraiment faire.

J'ai integre La Plateforme, et pendant ma formation j'ai effectue deux ans d'alternance chez Alltricks, le leader francais de l'e-commerce sport et cycle. Une grosse structure, une quarantaine de developpeurs cote backend, un site a fort trafic avec de vraies contraintes de production. Ca m'a appris ce que ca veut dire ecrire du code qui tient a l'echelle.

TrackR est ne de l'envie de concevoir quelque chose entierement a moi, de la conception jusqu'au deploiement. C'est ce projet que je presente dans ce dossier.



02 / PRESENTATION DU PROJET

TrackR

Portail client white-label pour freelances français

CONTEXTE DU MARCHÉ

La France comptait **4,8 millions de travailleurs indépendants** fin 2024, dont 2,9 millions d'auto-entrepreneurs en progression de 8,6 % sur un an (URSSAF, 2025). Parmi eux, les **freelances de services intellectuels** (conseil 24 %, communication et marketing 23 %, graphisme 20 %, tech et data 19 %) représentent le cœur de la cible TrackR (Malt & BCG, Freelancing in Europe 2024). Ce sont des indépendants qui facturent des clients, livrent des projets, et ont besoin d'une image professionnelle face à leurs clients.

En pratique, la gestion d'un client implique au minimum : email pour les contrats, Google Drive pour les documents, un outil de facturation (Wave, Stripe, ou un PDF manuel), Notion ou Trello pour le suivi, et WhatsApp pour les retours en cours de projet. Aucun de ces outils ne parle aux autres. Le client ne sait jamais où en est son projet, les factures se perdent dans les boîtes mail, et l'image perçue n'est pas celle d'un professionnel organisé.

PROBLÉMATIQUE

Comment permettre à un freelance français de centraliser toute la relation client, de la prospection jusqu'au paiement, dans un espace professionnel à son image, sans friction pour le client ?

En pratique, quatre problèmes reviennent systématiquement :

1. **Fragmentation des outils** : 5 à 7 applications par client, aucune cohérence pour le client final.
2. **Manque de visibilité** : le client ne sait pas où en est son projet sans relancer le freelance.
3. **Friction au paiement** : les factures se perdent dans les boîtes mail, les relances sont manuelles.
4. **Image peu professionnelle** : les outils génériques nuisent au positionnement du freelance auprès de ses clients.



UTILISATEURS CIBLES

Lucas, 29 ans Dev web freelance

TJM 550 EUR/j · SASU · 3-6 clients actifs

Besoin : que ses clients voient l'avancement sans le relancer.
Frustration : les paiements arrivent en retard parce que les factures se perdent dans les boites mail.

Karim, 34 ans Consultant transformation digitale

TJM 700 EUR/j · SASU · 2-3 missions longues (3-6 mois)

Besoin : espace professionnel pour les directions IT de grands comptes. Frustration : un Notion partage n'est pas à l'image d'un consultant senior qui facture 700 EUR par jour.

Mehdi, 31 ans Motion designer / DA

TJM 450 EUR/j · micro-entreprise · 3-5 clients

Besoin : partager les productions (videos, maquettes) et collecter des validations dans un seul espace. Frustration : les allers-retours par email font perdre la trace des décisions client.

Sophie, 35 ans Consultante UX

TJM 480 EUR/j · micro-entreprise · 2-4 clients

Besoin : image cohérente auprès des grandes entreprises qu'elle accompagne. Frustration : partager un Figma et un Google Drive ne reflète pas son positionnement.

Ines, 27 ans Rédactrice / content strategist

TJM 280 EUR/j · micro-entreprise · 6-10 clients

Besoin : tracker les livraisons et automatiser les relances de paiement. Frustration : passe plus de temps à relancer qu'à écrire en fin de mois.

Claire, 40 ans Chef de projet freelance

TJM 550 EUR/j · SASU · 3-6 projets en parallèle

Besoin : donner de la visibilité à plusieurs clients en même temps sans organiser des réunions. Frustration : le reporting manuel hebdomadaire prend une demi-journée que le kanban partagé remplacerait.



PROPOSITION DE VALEUR

TrackR est un portail client white-label conçu pour le cadre légal français. Les outils internationaux existants (SuiteDash, Assembly, Plutio) proposent des portails similaires mais ignorent le droit français de la facturation. Les outils français gèrent la conformité mais sans portail client. TrackR combine les deux.

Pour le freelance : ses factures sont conformes au droit français sans qu'il ait à vérifier chaque mention lui-même. Le client peut payer en ligne directement via Stripe, signer un devis, et suivre l'avancement de son projet depuis son espace. Moins de relances, moins d'allers-retours par email.

Pour le client : un espace à l'image du freelance (logo, couleurs, domaine), accessible sans compte ni mot de passe. L'accès se fait par magic-link (lien envoyé par email, valable 7 jours, usage unique). Le client retrouve ses documents, règle ses factures et suit l'avancement de son projet sans relancer le freelance.



ANALYSE CONCURRENTIELLE

Les outils internationaux (SuiteDash, Assembly ex-Copilot, Plutio) proposent un portail client white-label et du kanban, mais n'implémentent pas le cadre legal francais de la facturation : 16 mentions obligatoires (art. 289 CGI), immuabilite des factures (loi anti-fraude 2018), mention franchise TVA (art. 293B), numerotation sans trous. Les outils francais (Freebe, Facture.net) gerent la conformite mais sans portail client ni kanban partage. TrackR est concu specifiquement pour le droit francais, avec un portail white-label et un suivi de projet visible par le client.

Outil	Portail	White-label	Factu. FR	Kanban	Prix/mois
SuiteDash	Oui	Oui	Non	Oui	\$19-99
Assembly (ex-Copilot)	Excellent	Oui	Non	Non	\$29-399
Plutio	Oui	Oui	Non	Non	\$19
HoneyBook	Partiel	Partiel	Non	Non	\$36-109
Freebe	Non	Non	Oui	Non	11-45 EUR
Facture.net	Non	Non	Oui	Non	Gratuit-9 EUR
TrackR	Oui	Oui	Oui	Oui	TBD

PERIMETRE FONCTIONNEL

L'application couvre 9 modules fonctionnels :

- 01** Authentification freelance (JWT)
- 02** Configuration et branding white-label
- 03** Gestion des clients (statuts, magic-link)
- 04** Documents (upload PDF, visibilite)
- 05** Facturation (16 mentions legales, Stripe)
- 06** Tableau de bord (clients, projets, devis)
- 07** Kanban partage (vue client incluse)
- 08** Portail client (magic-link, paiement)
- 09** Notifications email (branding freelance)



03 / ORGANISATION DU PROJET

Methode et planification

METHODE DE TRAVAIL

Le projet a ete conduit en **developpement solo** sur une duree de 8 mois (novembre 2025 a juillet 2026). En l'absence d'equipe, la methode agile a ete adaptee : pas de standup ni de ceremonies formelles, mais une organisation en **sprints de deux semaines** avec des objectifs clairs, un journal de bord tenu a chaque sprint, et une revue a mi-parcours.

J'ai tenu un **journal de bord technique** a chaque sprint pour noter les decisions d'architecture, les problemes rencontres et comment ils ont ete resolus. Les commits suivent la convention **Conventional Commits** (`feat:`, `fix:`, `chore:`, `ci:`) directement sur `main` : en solo, une branche `develop` n'apporte rien et complique l'historique inutilement.

Sequencement API-first. Le backend a ete entierement construit et stabilise (sprints S0 a S8) avant de toucher au frontend (S9 a S18). La raison est simple : si les contrats de donnees (types de reponses, codes HTTP, structure JSON) bougent en cours de route, le frontend doit suivre. En fixant l'API d'abord, le frontend consomme quelque chose de stable.

CONVENTION DE COMMITS

feat:	Nouvelle fonctionnalite (ex. : <code>feat: Stripe webhook local</code> , <code>feat: white-label portail</code>). Declenche le pipeline CI complet et le deploy en production.
fix:	Correction de bug (ex. : <code>fix(stripe): use PORTAIL_BASE_URL</code> , <code>fix(auth): redirect apres reset</code>). Meme pipeline que <code>feat:</code> .
chore:	Changement sans impact fonctionnel (nettoyage UX, suppression code mort, mise a jour config).
ci:	Modifications du pipeline GitHub Actions (inject secrets, correction build context, retrigger deploy).



PLANNING : 18 SPRINTS

Sprint	Objectif	Periode
S0	Cadrage, MCD, infrastructure Docker + CI/CD	Nov. 2025
S1	Authentification freelance (JWT, inscription, connexion)	Nov. 2025
S2	Clients et projets (CRUD, statuts)	Dec. 2025
S3	Portail client : magic-link, statuts prospect/actif, documents	Dec. 2025
S4	Branding white-label et informations legales (SIRET, TVA)	Jan. 2026
S5	Facturation conforme (16 mentions CGI, PDF DomPDF)	Jan. 2026
S6	Stripe Connect OAuth (paiement, webhooks)	Fev. 2026
S7	Kanban partage (colonnes, taches, vue client)	Fev. 2026
S8	Notifications brandees et emails asynchrones (Messenger)	Mar. 2026
S9	Foundation frontend React (routing, auth, layout)	Mar. 2026
S10	Design system et shell applicatif (ShadCN, sidebar)	Avr. 2026
S11	Dashboard et module clients (frontend)	Avr. 2026
S12	Projets et Kanban (frontend, drag-and-drop)	Mai 2026
S13	Factures (frontend, generation, envoi)	Mai 2026
S14	Documents et parametres (upload, profil)	Mai 2026
S15	Portail client (frontend complet, magic-link, paiement)	Juin 2026
S16	Devis freelance et notifications in-app	Juin 2026
S18	Refonte auth, conformite legale, corrections UX	Juil. 2026



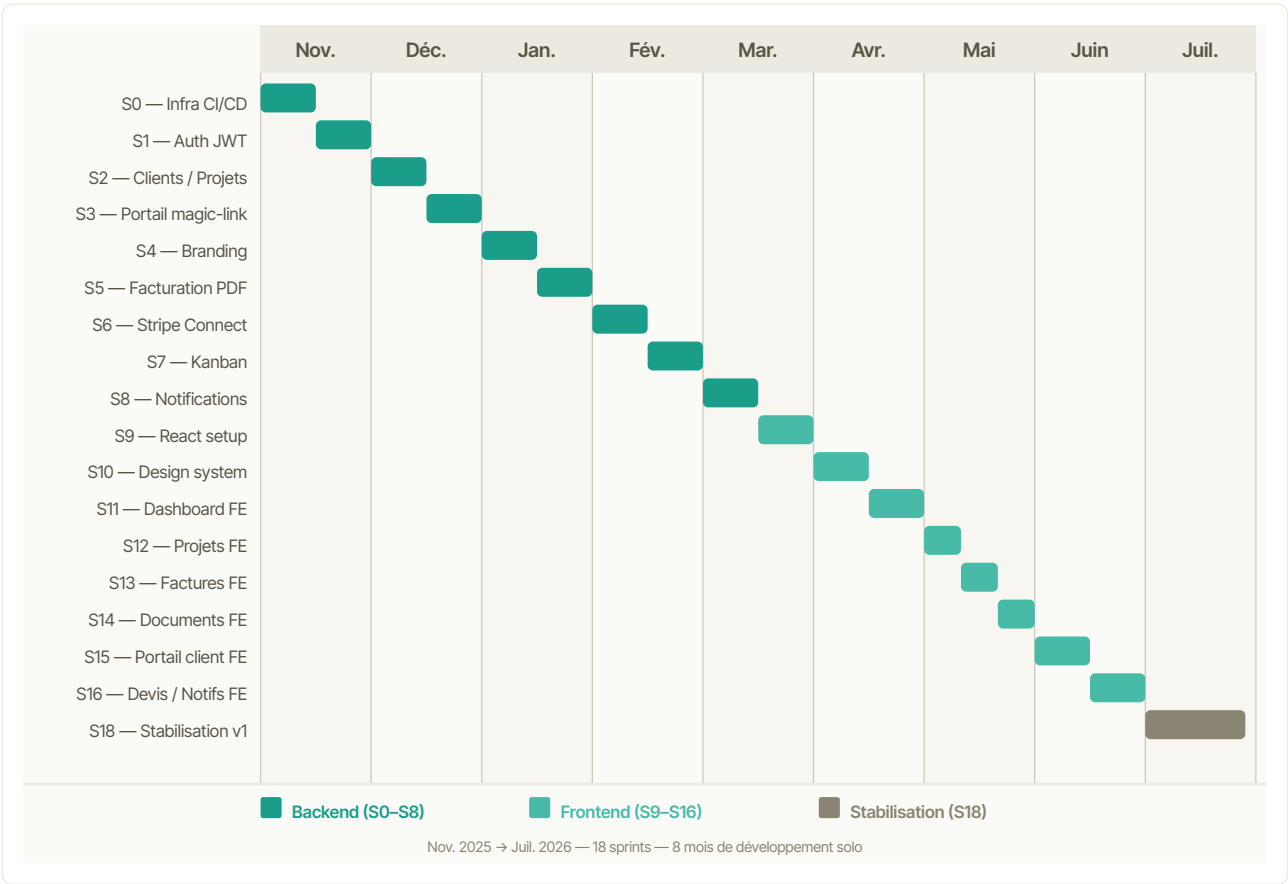
PLANIFICATION : JALONS ET LIVRABLES

La planification couvre 8 mois de développement solo de novembre 2025 à juillet 2026. Les jalons correspondent aux grandes transitions du projet : conception, socle technique, fonctionnalités cœur, portail client, et finalisation.

Periode	Jalon	Livrable cle
Nov. 2025	J0 : Conception et infrastructure	MCD, 25 US, pipeline CI vert, Docker multi-stage
Nov.-Dec. 2025	J1 : Socle authentification	JWT, magic-link, double firewall Symfony, tests d'integration
Dec. 2025-Jan. 2026	J2 : Modules metier coeur	CRUD clients/projets, branding, infos légales
Jan.-Fev. 2026	J3 : Facturation et paiement	16 mentions CGI, PDF DomPDF, Stripe Connect OAuth
Fev.-Mar. 2026	J4 : Kanban et notifications	Drag-and-drop, colonnes partages, emails brandes
Mar.-Avr. 2026	J5 : Frontend : socle	React 19, routing, design system ShadCN, shell
Avr.-Mai 2026	J6 : Frontend : modules freelance	Dashboard, clients, projets, factures, devis
Mai-Juin 2026	J7 : Portail client complet	Portail brande, paiement Stripe, Kanban client
Juin-Juil. 2026	J8 : Finalisation	Devis frontend, notifications in-app, corrections UX, CDA



GANTT : PLANNING DES 18 SPRINTS



METRIQUES DU PROJET

18
sprints

150+
tests PHPUnit

30+
endpoints API

19
maquettes



USER STORIES : PERIMETRE MVP

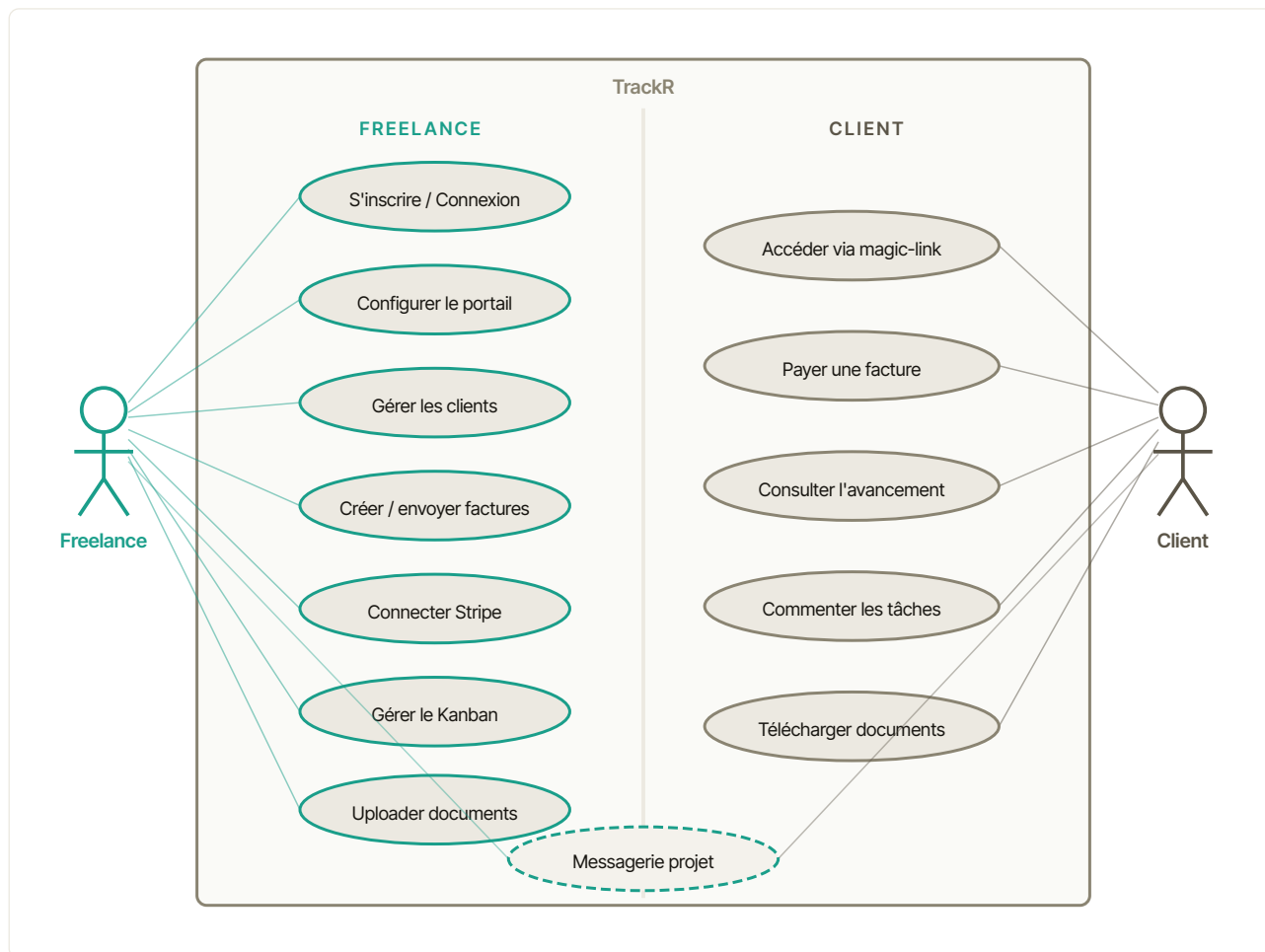
Les 25 User Stories ont été priorisées selon la méthode MoSCoW et découpées en 7 epics.

ID	En tant que... je veux...	Criteres d'acceptation	Priorite
US-01	Freelance : m'inscrire	POST /register 201 + JWT, Entreprise + User creés	Must
US-02	Freelance : connecter Stripe	OAuth Connect, stripe_account_id stocké, charges_enabled vérifié	Must
US-03	Freelance : configurer le branding	Logo upload, couleur hex, nom portail, instructions CNAME	Must
US-04	Freelance : renseigner infos légales	SIRET (14 chiffres), TVA/293B, IBAN, forme juridique	Must
US-05	Freelance : créer un client et l'inviter	Email magic-link envoyé, portail accessible, statut prospect	Must
US-06	Freelance : gérer un prospect avec accès limité	Sections verrouillées côté client, upgrade vers actif	Must
US-07	Client : accéder au portail via lien email	Token 64 chars, TTL 7j, usage unique, page renvoi si expire	Must
US-08	Client : vue d'ensemble du portail	Documents, projets, factures à payer, badges statuts	Must
US-09	Client : demander un nouveau lien expire	Page publique /portail/acces avec champ email	Must
US-10	Freelance : uploader des documents	PDF max 10 Mo, catégorie, toggle visible_client / prospect	Must
US-11	Client : télécharger ses documents	Liste filtrée (visible_client = true), téléchargement direct	Must
US-12	Freelance : historique d'accès aux documents	Log consultations (client_id, document_id, accessed_at)	Should
US-13	Freelance : créer une facture légale	16 mentions CGI, numéro FAC-YYYY-NNN, snapshot_client JSONB	Must
US-14	Freelance : envoyer la facture	Stripe Payment Intent créée, lien paiement dans portail, email client	Must
US-15	Freelance : rappels automatiques	3 déclencheurs (avant / à / après échéance), Symfony Messenger	Must
US-16	Client : payer par carte bancaire	Stripe Checkout card, redirect post-paiement vers portail	Must
US-17	Freelance : factures payées non modifiables	Statut payée = lecture seule, PDF stocké définitivement	Must
US-18	Freelance : créer des projets	Statuts prospect/actif/pause/termine/archive, date_debut/fin	Must
US-19	Client : voir l'avancement de son projet	Badge statut, dates, % tâches terminées	Must
US-20	Freelance : gérer un Kanban par projet	CRUD colonnes + tâches, drag-and-drop, colonnes par défaut	Must
US-21	Client : voir le Kanban et valider les tâches	Vue lecture + commentaires sur tâches "En test client"	Must
US-22	Client : envoyer un message depuis le portail	Fil commentaires par projet, notification email freelance	Must
US-23	Freelance : répondre aux messages clients	Réponse depuis fiche client, notification email client	Must
US-24	Client : emails brandés pour événements clés	Templates avec logo + couleur primaire du freelance	Must
US-25	Freelance : notifié quand client paie/commente	Webhook Stripe, commentaire, notification in-app + email	Must



DIAGRAMME DE CAS D'UTILISATION (UML)

Deux acteurs principaux interagissent avec le système : le **Freelance** (cote administration) et le **Client** (cote portail white-label). La messagerie projet est le seul cas d'utilisation partagé.





04 / CONCEPTION

Architecture et modelisation

MODELE DE DONNEES

La base de donnees a ete modelisee avec la methode **Merise** (MCD, puis MLD, puis modele physique PostgreSQL). Elle compte **16 entites** organisees autour du concept d'Entreprise (tenant isolee correspondant a un freelance).

Entreprise

Tenant racine, un freelance = une entreprise

ClientAcces

Token magic-link (64 chars, TTL 7j, usage unique)

Projet

Associe a un client, statuts multiples

Document

PDF upload, visibilite client/prospect configurable

Devis

Proposition commerciale avec statut et PDF

ActivityLog

Audit log consultations documents (client_id, doc_id)

User

Compte freelance, rattache a une Entreprise

Facture

16 mentions legales, snapshot_client JSONB immuable

Colonne

Kanban : colonnes ordonnees, position et visibilite client

StripeConfig

Compte Stripe Connect du freelance (account_id)

Commentaire

Fil de messages par projet (freelance / client)

Client

Client du freelance, statut prospect/actif/archive

LigneFacture

Lignes de prestation (qte, prix HT, total HT)

Tache

Tache kanban, assignable freelance/client, date echeance

Branding

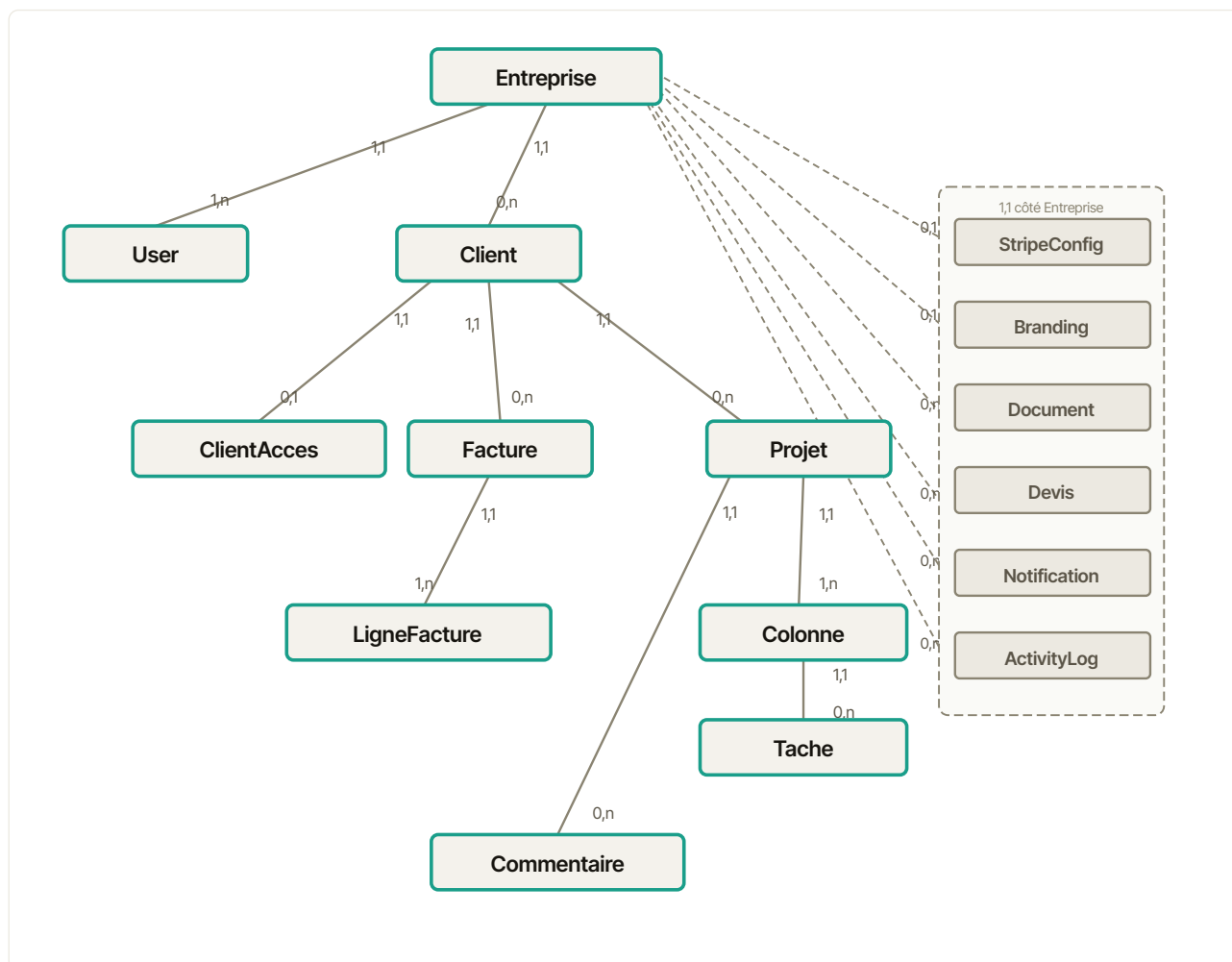
Logo, couleur primaire, nom du portail

Notification

Notifications in-app (paiement, commentaire, tache)



MCD : MODELE CONCEPTUEL DE DONNEES (MERISE)



MCD Merise : 16 entites, cardinalites simplifiees. La zone en pointilles regroupe les 6 modules satellites d'Entreprise (StripeConfig, Branding, Document, Devis, Notification, ActivityLog). Commentaire est lie a Projet (0,n), hors zone.

Decisions de conception notables :

1. snapshot_client JSONB sur Facture : denormalisation volontaire. Les coordonnees client peuvent changer apres emission. Le droit francais exige que la facture reflète les coordonnees exactes au moment de l'emission.
2. Token magic-link stocke en clair (pas de hash) : la revocation immediate en cas de compromis serait impossible avec un token hache. Duree de vie courte (7j) et usage unique limitent le risque.
3. Numérotation facture via sequence PostgreSQL par (entreprise_id, annee) : garantit l'absence de trous, exigence legale de la loi anti-fraude 2018.



ARCHITECTURE LOGICIELLE

L'application suit une **architecture en couches** cote backend (Symfony) et une architecture **composants** cote frontend (React).

Controller	Point d'entree HTTP. Validation, deserialisation, appel du service ou du repository, serialisation JSON.
Service	Logique metier pure (FactureService, DevisService, NotificationService). Injecte via le conteneur Symfony.
Repository	Requetes Doctrine. Chaque entite dispose de son repository (findByEntreprise, findBy-Token...).
Entity	Entites Doctrine mappees sur PostgreSQL. Pas d'annotations metier dans les entites (separation des preoccupations).
Message/Handler	Symfony Messenger : taches asynchrones (emails, PDF). Le handler consomme via Redis.



ENTITES : CHAMPS ET CONTRAINTES

Les 16 entites du modele physique sont decrites ci-apres avec leurs champs, types et contraintes principales.

Entreprise

Champ	Type SQL	Contrainte / Note
id	UUID	PK, gen_random_uuid()
nom	VARCHAR(255)	NOT NULL
siret	VARCHAR(14)	nullable, valide en PHP (14 chiffres)
tva	VARCHAR(50)	nullable (ex : FR12345678901 ou 293B)
adresse / ville / pays	VARCHAR	nullable
forme_juridique	VARCHAR(100)	EI, SASU, SARL, SAS...
iban / bic	VARCHAR	nullable, utilises dans le PDF facture
created_at	TIMESTAMP	NOT NULL

User (Utilisateur)

Champ	Type SQL	Contrainte / Note
id	UUID	PK
email	VARCHAR(255)	UNIQUE NOT NULL
password_hash	VARCHAR(255)	NOT NULL, argon2id via Symfony
email_verified_at	TIMESTAMP	nullable, null = en attente
entreprise_id	UUID FK	NOT NULL REFERENCES entreprise(id)
created_at	TIMESTAMP	NOT NULL

Client

Champ	Type SQL	Contrainte / Note
id	UUID	PK
entreprise_id	UUID FK	NOT NULL (isolation tenant)
nom / email	VARCHAR	NOT NULL
telephone / adresse / ville / pays	VARCHAR	nullable
created_at	TIMESTAMP	NOT NULL



ClientAcces

Champ	Type SQL	Contrainte / Note
id	UUID	PK
client_id + entreprise_id	UUID FK	NOT NULL, UNIQUE ensemble
token	VARCHAR(128)	hex 64 octets, usage unique
token_expire_at	TIMESTAMP	NOW() + 7 jours
token_used_at	TIMESTAMP	nullable, set au premier clic
session_token	VARCHAR(256)	nullable, cree apres usage magic-link
session_expire_at	TIMESTAMP	nullable, NOW() + 30 jours
statut	ENUM	prospect actif pause

Facture

Champ	Type SQL	Contrainte / Note
id	UUID	PK
entreprise_id / client_id	UUID FK	NOT NULL
numero	VARCHAR(20)	UNIQUE, FAC-YYYY-NNN via sequence PostgreSQL
statut	ENUM	brouillon envoyee vue payee en_retard annulee
date_emission / date_echeance	DATE	NOT NULL / nullable
snapshot_client	JSONB	NOT NULL, fige a l'envoi (art. 289 CGI)
montant_ht / tva / ttc	DECIMAL(12,2)	calculs par FactureService
taux_tva	DECIMAL(5,2)	0 / 5.5 / 10 / 20 (%)
stripe_payment_url	VARCHAR	nullable, lien Stripe Checkout

LigneFacture

Champ	Type SQL	Contrainte / Note
id	UUID	PK
facture_id	UUID FK	NOT NULL
description	TEXT	NOT NULL
quantite	DECIMAL(10,3)	ex : 2.5 jours
prix_unitaire_ht	DECIMAL(12,2)	NOT NULL
total_ht	DECIMAL(12,2)	quantite x prix_unitaire_ht
position	INTEGER	NOT NULL, ordre d'affichage



Projet

Champ	Type SQL	Contrainte / Note
id	UUID	PK
entreprise_id / client_id	UUID FK	NOT NULL / nullable (ON DELETE SET NULL)
titre	VARCHAR(255)	NOT NULL
statut	ENUM	actif pause termine archive
date_debut / date_fin	DATE	nullable
tarif_journalier	DECIMAL(10,2)	nullable (EUR)

Colonne / Tache (Kanban)

Champ	Type SQL	Contrainte / Note
colonne.id	UUID	PK, REFERENCES projet(id)
colonne.nom	VARCHAR(100)	Backlog, En cours, En test client...
colonne.position	INTEGER	ordre d'affichage
colonne.visible_client	BOOLEAN	DEFAULT true
tache.id	UUID	PK, REFERENCES colonne(id) + projet(id)
tache.titre	VARCHAR(255)	NOT NULL
tache.assignee	ENUM	freelance client
tache.date_echeance	DATE	nullable

Document / StripeConfig / Branding / Devis

Champ	Type SQL	Contrainte / Note
document.categorie	ENUM	cgv contrat devis facture livrable autre
document.visible_client	BOOLEAN	DEFAULT false
document.taille_octets	BIGINT	capturee avant move() (comportement UploadedFile)
stripe_config.stripe_account_id	VARCHAR(255)	acct_XXXX, mise a jour via webhook
stripe_config.charges_enabled	BOOLEAN	verifie avant affichage du bouton paiement
branding.couleur_primaire	VARCHAR(7)	format hex, injectee en variable CSS
branding.nom_portail	VARCHAR(255)	affiche en lieu et place de "TrackR"
devis.statut	ENUM	brouillon envoye vu signe refuse expire



Commentaire

Champ	Type SQL	Contrainte / Note
id	UUID	PK
projet_id	UUID FK	NOT NULL REFERENCES projet(id) ON DELETE CASCADE
auteur_type	ENUM	freelance client
auteur_id	VARCHAR(36)	UUID polymorphe (User ou ClientAcces, sans FK)
auteur_nom	VARCHAR(150)	snapshot du nom au moment de l'envoi
contenu	TEXT	NOT NULL
created_at	TIMESTAMP	NOT NULL

Notification / ActivityLog

Champ	Type SQL	Contrainte / Note
notification.entreprise_id	UUID FK	NOT NULL, destinataire freelance
notification.type	VARCHAR(20)	paiement_recu commentaire devis_signe portail_ouvert livrable_approuve
notification.titre	VARCHAR(120)	NOT NULL
notification.lu	BOOLEAN	DEFAULT false
activity_log.client_id	UUID FK	NOT NULL REFERENCES client(id)
activity_log.type	VARCHAR(30)	portail_ouvert facture_vue document_telecharge commentaire_poste
activity_log.meta	JSON	nullable, contexte supplementaire (doc_id, facture_id...)



SCHEMA SQL : EXTRAITS DDL

Les migrations Doctrine generent du SQL pur (pas de doctrine:schema:update). Extraits des tables principales :

```
CREATE TABLE entreprise (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nom         VARCHAR(255)      NOT NULL,
    siret       VARCHAR(14),
    tva         VARCHAR(50),
    adresse     VARCHAR(500),
    forme_juridique VARCHAR(100),
    iban        VARCHAR(34),
    bic         VARCHAR(11),
    created_at  TIMESTAMP        NOT NULL DEFAULT NOW()
);

CREATE TABLE utilisateur (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email       VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    email_verified_at TIMESTAMP,
    entreprise_id UUID NOT NULL REFERENCES entreprise(id) ON DELETE CASCADE,
    created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_user_email ON utilisateur(email);

CREATE TABLE client_acces (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    client_id   UUID NOT NULL REFERENCES client(id) ON DELETE CASCADE,
    entreprise_id UUID NOT NULL REFERENCES entreprise(id) ON DELETE CASCADE,
    token       VARCHAR(128) NOT NULL,
    token_expire_at TIMESTAMP NOT NULL,
    token_used_at TIMESTAMP,
    session_token VARCHAR(256),
    session_expire_at TIMESTAMP,
    statut      VARCHAR(20) NOT NULL DEFAULT 'prospect',
    UNIQUE (client_id, entreprise_id)
);

-- Index partiel : uniquement les sessions actives
CREATE INDEX idx_acces_session ON client_acces(session_token)
    WHERE session_token IS NOT NULL;

-- Sequence pour la numerotation des factures par entreprise et annee
CREATE SEQUENCE IF NOT EXISTS facture_seq_2026;

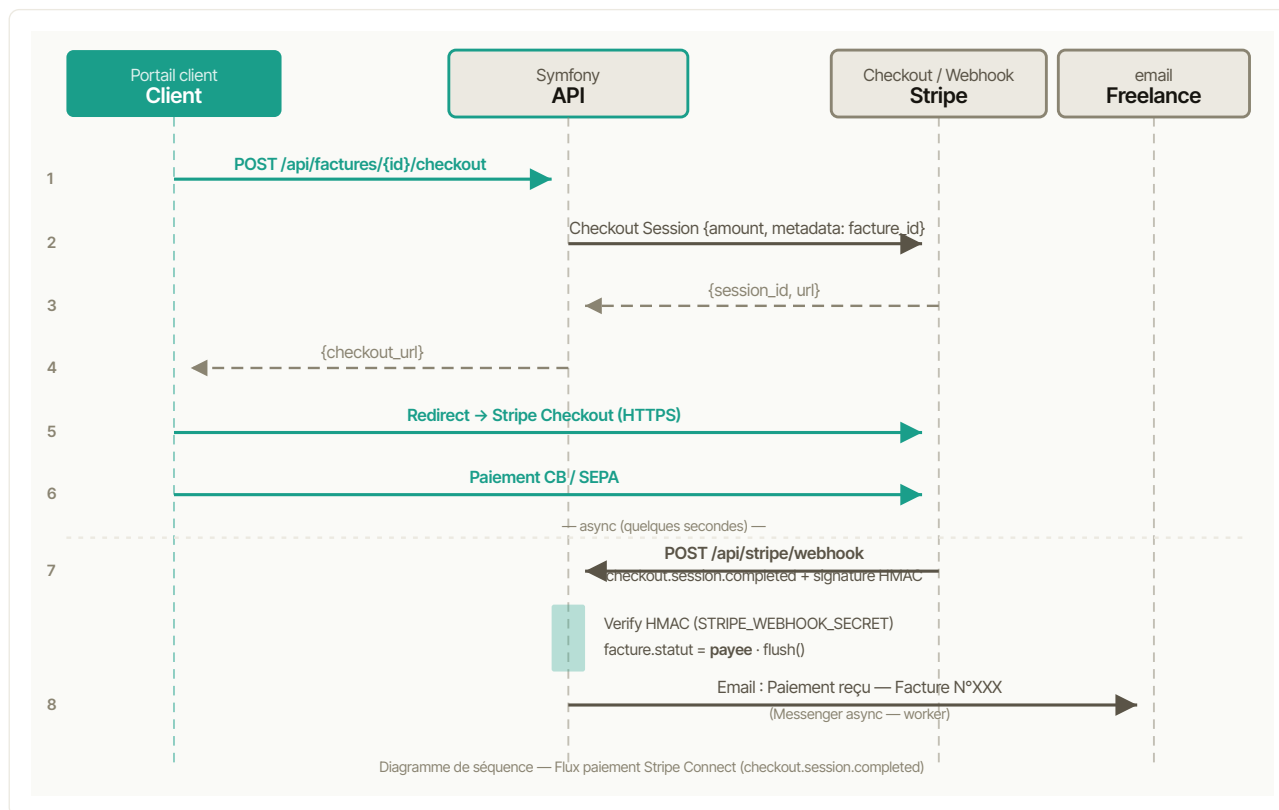
CREATE TABLE facture (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entreprise_id UUID NOT NULL REFERENCES entreprise(id),
    client_id   UUID NOT NULL REFERENCES client(id),
    numero      VARCHAR(20) UNIQUE NOT NULL,
    statut      VARCHAR(20) NOT NULL DEFAULT 'brouillon',
    date_emission DATE NOT NULL,
    date_echeance DATE,
    snapshot_client JSONB NOT NULL,
    montant_ht  DECIMAL(12,2) NOT NULL DEFAULT 0,
    taux_tva    DECIMAL(5,2) NOT NULL DEFAULT 20,
    montant_tva DECIMAL(12,2) NOT NULL DEFAULT 0,
    montant_ttc DECIMAL(12,2) NOT NULL DEFAULT 0,
    stripe_payment_url VARCHAR(500),
    created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_facture_entreprise_statut ON facture(entreprise_id, statut);
```



FLUX PRINCIPAUX : DIAGRAMMES DE SEQUENCE

Flux visuel : Paiement Stripe Connect



Flux 1 : Inscription freelance (POST /api/auth/register)

Acteur	Action
Frontend	POST {email, password, nom_entreprise}
Controller	Validation email, longueur password (≥ 8), unicite
Service	new Entreprise(nom), new User(email), hashPassword argon2id
Doctrine	persist(entreprise) + persist(user) + flush() : transaction atomique
Mailer	Dispatch SendVerificationEmailMessage (async Messenger)
Controller	Retour 201 {message: "verification_sent"}

Flux 2 : Authentification JWT (POST /api/auth/login)

Acteur	Action
Frontend	POST {email, password}
RouterListener	Trouve la route stub /api/auth/login (priorite 32)
FirewallListener	Intercepte via json_login (priorite 8)
UserProvider	findByEmail() : verifie email_verified_at $\neq$ null
PasswordHasher	verifyPassword(hash, input) en argon2id
LexikJWT	Genere access token RS256 (TTL 7 jours)
Firewall	Retour 200 {token}



Flux 3 : Acces portail client via magic-link

Acteur	Action
Freelance	POST /api/clients/{id}/inviter
Controller	Genere token hex 64 octets (random_bytes(32)), TTL 7j
Mailer	Email HTML avec lien /portail/auth/{token} (branding freelance)
Client	Clique le lien : GET /api/portail/auth/{token}
Controller	findByToken(), verifie expiration et token_used_at = null
ClientAcces	useToken() : token_used_at = now(), genere session_token 30j
Controller	Retour JSON {session_token, statut}
Frontend portail	Stocke cookie, redirige vers /portail

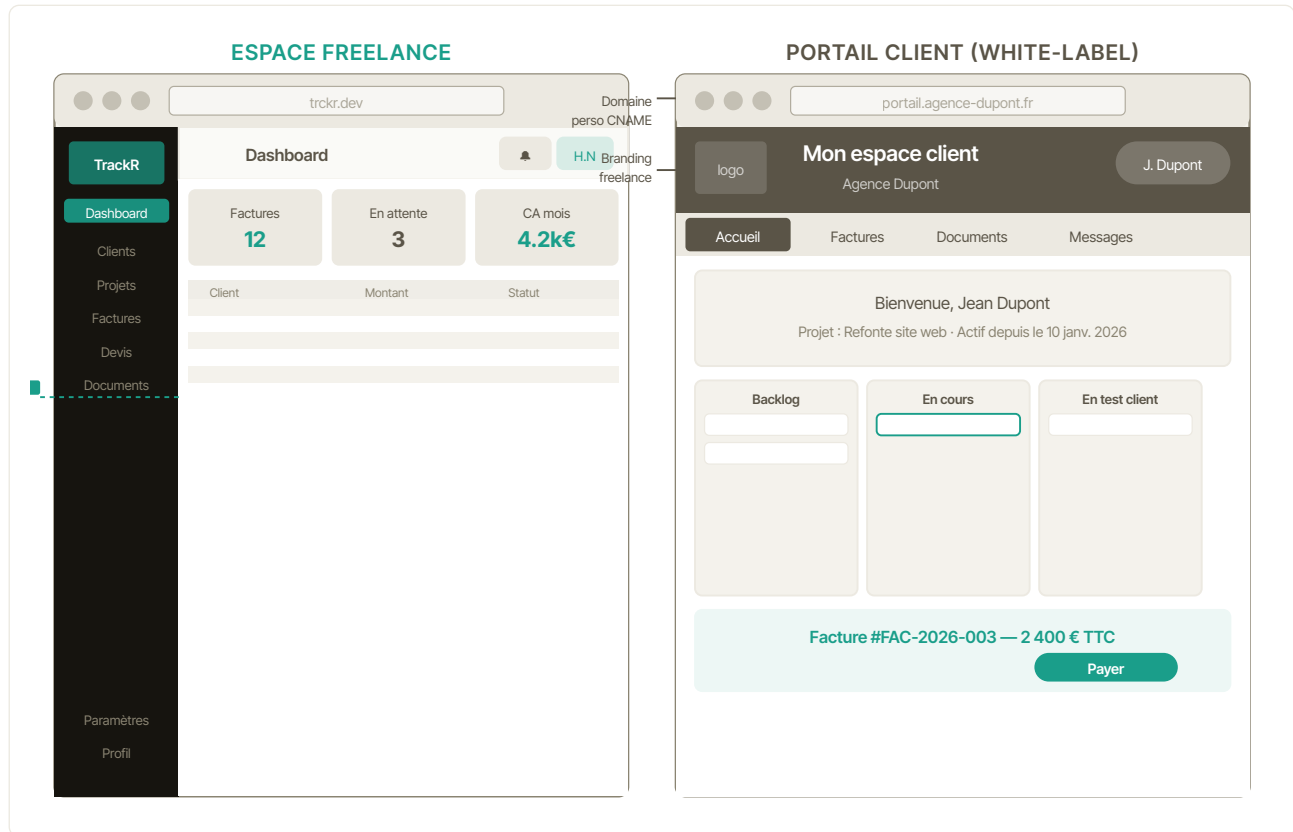
Flux 4 : Paiement facture (Stripe Connect)

Acteur	Action
Client portail	Clic "Payer" sur la facture
Backend	POST /api/factures/{id}/checkout : cree Stripe Checkout Session
Stripe	Redirige vers page de paiement Stripe (carte bancaire)
Client	Saisit ses coordonnees CB
Stripe	POST /api/stripe/webhook : checkout.session.completed
Backend	Verifie signature HMAC (STRIPE_WEBHOOK_SECRET)
Backend	facture.statut = payee, flush(), email de notification freelance



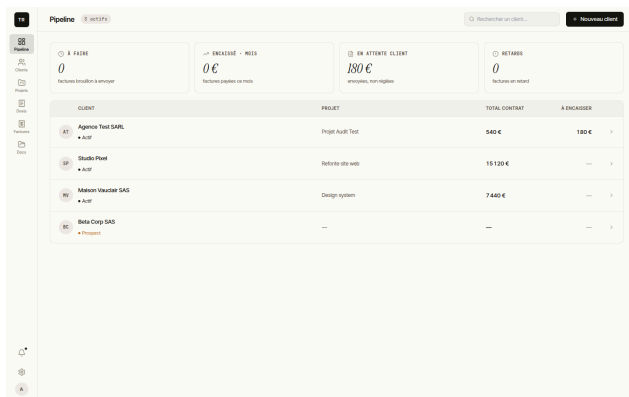
ZONING : STRUCTURE DES INTERFACES

L'application présente deux interfaces distinctes : l'espace d'administration du freelance (sidebar fixe, navigation par modules) et le portail client white-label (branding freelance, accès sans compte, navigation simplifiée).

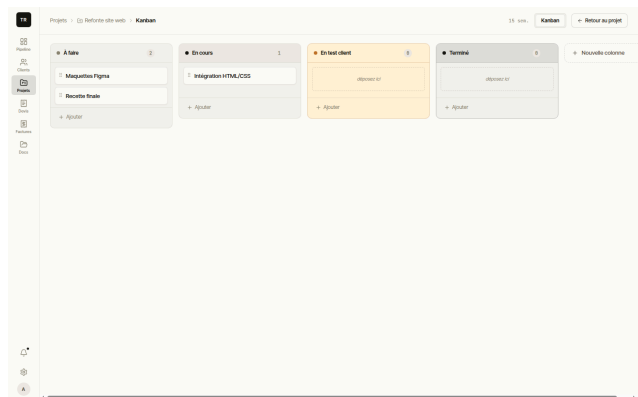




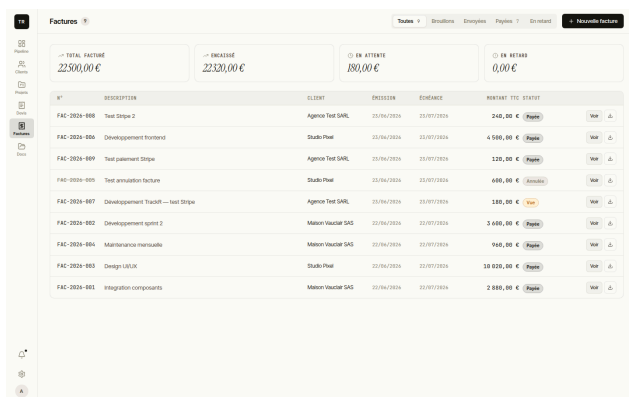
MAQUETTES : INTERFACES PRINCIPALES



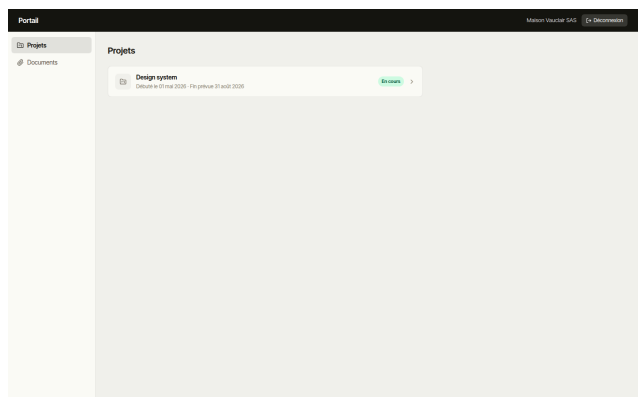
Dashboard freelance : vue d'ensemble



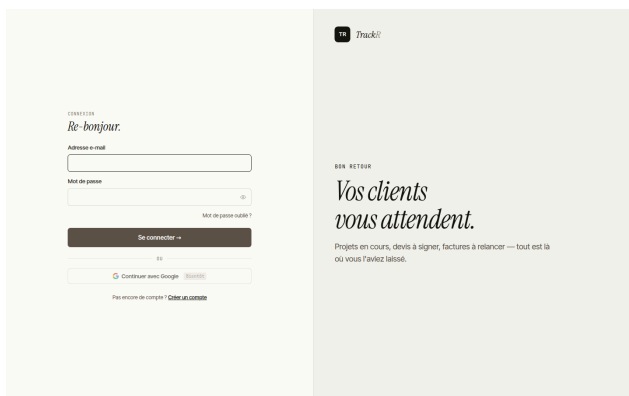
Kanban partage : colonnes et tâches



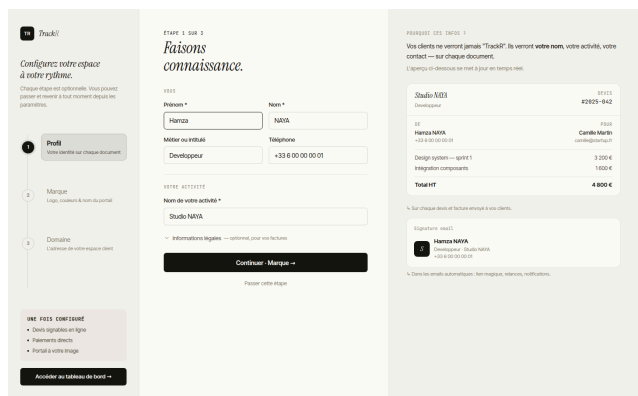
Module facturation : liste et statuts



Portail client : vue acces magic-link



Connexion freelance : email et password



Onboarding : configuration initiale



Informations

Refonte site web

33%

Facturation :

N°	Montant	Date	Statut
FAC-2026-006	4 500 €	25 Jan 2026	Payé
FAC-2026-005	600 €	25 Jan 2026	Payé
FAC-2026-003	18 820 €	25 Jan 2026	Payé

Fiche client : historique, projets, factures

Projets

3 TOTAL PROJETS, 3 ACTIFS, 0 EN PAYSSE, 0 TERMINEES

PROJET	CLIENT	STATUT	PROGRESSION	BUDGET
PROJ-2026-001	Agence Test SASL	ACTIF	100%	550 €
PROJ-2026-002	Studio Phil	ACTIF	100%	550 €
PROJ-2026-003	Maison Vauclair SAS	ACTIF	100%	600 €

Liste des projets : statuts et filtres

Devis

N°	Description	CLIENT	MONTANT TTC	STATUT
D-2026-001	Developpement site e-commerce	Studio NAYA	0,00 €	Brouillon
D-2026-002	Refonte identite visuelle	Studio NAYA	0,00 €	Brouillon
D-2026-003	Maintenance mensuelle	Studio NAYA	0,00 €	Brouillon

Module devis : liste et statuts

DEVIS

D-2026-001

DESCRIPTION	QUOTE	Prix HT	TOTAL HT
UX/UI Design (Inauguration)	3	0,00 €	0,00 €
Developpement frontend React	8	0,00 €	0,00 €
Integration API backend	5	0,00 €	0,00 €
Total HT		0,00 €	0,00 €
TVA (20%)			0,00 €
Total TTC			0,00 €

Detail devis : lignes et envoi client

Documents

N°	DATE	TAILLE	VISIBILITE	DATE
----	------	--------	------------	------

Vous n'avez pas encore de documents.

Gestion documents : upload et visibilité

Notifications

Titre	Contenu	Date
Devis signé — D-2026-006	Beta Corp SAS a signé le devis "Test email signature"	26 Jan
Beta Corp SAS a ouvert le portail	Portail ouvert	26 Jan
Beta Corp SAS a ouvert le portail	Portail ouvert	26 Jan
Beta Corp SAS a ouvert le portail	Portail ouvert	26 Jan
Devis signé — D-2026-004	Maison Vauclair SAS a signé le devis "Signet P"	26 Jan
Maison Vauclair SAS a ouvert le portail	Portail ouvert	26 Jan
Nouveau commentaire de Agence Test SASL	Nouveau commentaire	26 Jan
Agence Test SASL a ouvert le portail	Portail ouvert	26 Jan
Paiement reçu — FAC-2026-009	Facture de 1212 € réglée	26 Jan
Agence Test SASL a ouvert le portail	Portail ouvert	26 Jan
Agence Test SASL a ouvert le portail	Portail ouvert	27 Jan
Agence Test SASL a ouvert le portail	Portail ouvert	27 Jan

Notifications in-app : centre de notifications



PARAMETRES

- Mon profil
- Entreprise
- Branding
- Stripe
- Sécurité

Mon profil

Informations qui apparaissent sur vos documents et emails clients.

Prénoms: HAMZA, Nom: NAYA

Mettre ou modifier:

Téléphone:

Adresse e-mail:

Pour changer votre email, contactez le support.

[Sauvegarder](#)

Parametres : profil et infos legales

PARAMETRES

- Mon profil
- Entreprise
- Branding
- Stripe
- Sécurité

Sécurité

Changez votre mot de passe et l'accès à votre compte.

Mot de passe

Mot de passe actuel:

Nouveau mot de passe:

Minimum 8 caractères.

Confirmer le nouveau mot de passe:

[Modifier le mot de passe](#)

Authentification à deux facteurs

Statut: Non activé [Activer la 2FA](#)

Sessions actives

Cette session: Navigation actuelle [Déconnecter](#)

Securite : sessions actives et 2FA

Devis / Nouveau devis

1016NKT285

Client:

Projet:

Objet:

LIGNES

DESCRIPTION	QTE	PU HT (€)	TOTAL HT
Description de la prestation	1	0	0,00 €

[Ajouter une ligne](#)

DÉTAILS

Taux TVA: Date d'expiration:

[Annuler](#) [Sauvegarder](#) [Déconnecter & renvoyer](#)

Nouveau devis : formulaire lignes de prestation

Factures / FAC-2020-003 / [Ajouter](#)

Audit Studios SAS

FACTURE # Studio Pixel

Projet: Redesign site web

DESCRIPTION	QTE	PU HT	TOTAL HT
Design UI/UX	1	3 000,00 €	3 000,00 €
Développement frontend	1	1 700,00 €	1 700,00 €
Sous-total HT		4 700,00 €	4 700,00 €
TVA 20%			940,00 €
Total TTC			5 640,00 €

Paiement à 30 jours à compter de la date d'émission. En cas de retard de paiement, des pénalités au taux légal en vigueur seront appliquées, ainsi qu'une indemnité forfaitaire de recouvrement de 40 € par L. 441-10 du Code de commerce.

ATTENTION

[Payer](#)

1016NKT285
FAC-2020-003
ES-EM-2020-0006
CLIENT
Studio Pixel
Pixel
Ref: Audit site web

Detail facture : statut et lien paiement Stripe



05 / DEVELOPPEMENT BACKEND

API Symfony 7.2

CHOIX TECHNIQUES

Le backend est developpe avec **Symfony 7.2** en PHP 8.4. J'ai prefere des controllers manuels a API Platform : chaque etape du flux HTTP est ecrite explicitement dans le code (deserialisation, validation, logique metier, persistance, serialisation). C'est plus verbeux, mais tout est lisible et justifiable sans avoir a connaitre les internals d'un bundle.

Symfony 7.2	Framework principal. Composants utilises : HttpKernel, Security, Messenger, Mailer, Scheduler, Serializer, Validator.
Doctrine ORM	Mapping entites/tables. Migrations SQL pures. Repositories avec requetes DQL specifiques.
PostgreSQL 17	UUID natif, JSONB pour snapshot_client, sequences pour numerotation, index partiels.
Redis 7.4	File de messages Symfony Messenger (emails async, PDF generation).
DomPDF	Generation des PDF factures et devis (rendu HTML vers PDF, polices embarquees).

AUTHENTIFICATION

Freelance (JWT) : authentification classique email/mot de passe via LexikJWTAuthenticationBundle. Le token expire au bout de 7 jours (`token_ttl: 604800`). L'inscription cree l'Entreprise et le premier User en une seule transaction : si l'un echoue, les deux sont annules.

Client portail (magic-link) : le client n'a pas de compte. Le freelance lui envoie un lien unique valable 7 jours. Au premier clic, le lien est invalide et une session de 30 jours est creee. Si le lien a expire, le client peut en demander un nouveau depuis la page `/portail/acces` sans passer par le freelance.

FACTURATION LEGALE

La facturation respecte les **16 mentions obligatoires de l'article 289 du CGI** et les regles de la **loi anti-fraude 2018** :

1. Numerotation sequentielle sans trous (`FAC-YYYY-NNN`) via sequence PostgreSQL par entreprise et par annee.
2. Immuabilite : une facture envoyee ne peut plus etre modifiee ni supprimee (statut machine a etats).
3. `snapshot_client` JSONB : les coordonnees du client sont figees au moment de l'envoi.
4. Gestion TVA : 0% (art. 293B), 5,5%, 10%, 20%. Mention automatique si franchise.
5. Penalites de retard et indemnite forfaitaire de recouvrement (40 EUR LME) incluses dans les CGV.



Les 16 mentions obligatoires (article 289 CGI) sont vérifiées lors de la génération du PDF :

1	Numero unique sequentiel (FAC-YYYY-NNN)	2	Date d'emission
3	Date ou periode de realisation	4	Nom, adresse, SIRET + forme juridique vendeur
5	Numero TVA intracommunautaire vendeur	6	Nom et adresse de l'acheteur
7	Description precise des prestations	8	Prix unitaire hors taxes
9	Quantite ou nombre de jours	10	Total HT par ligne de prestation
11	Total HT global (somme des lignes)	12	Taux de TVA applicable (ou mention 293B)
13	Montant de TVA correspondant	14	Total TTC
15	Conditions de paiement (delai, modes acceptes)	16	Penalites de retard + indemnite forfaitaire 40 EUR (LME)

Stripe Connect OAuth : le freelance connecte son propre compte Stripe via un flux OAuth (redirect vers Stripe, callback avec code, échange contre `stripe_account_id`). TrackR n'est jamais intermédiaire de paiement : les fonds arrivent directement sur le compte bancaire du freelance. Le paiement côté client s'effectue via Stripe Checkout (carte bancaire).

CONFORMITE RGPD

TrackR traite des données de deux types d'utilisateurs : les freelances qui ont un compte, et leurs clients qui accèdent au portail sans s'inscrire. Les choix de conception tiennent compte des deux.

- 1. Minimisation des données.** Les champs collectés à l'inscription (email, nom entreprise) sont strictement nécessaires. Le numéro de téléphone et l'adresse du client sont optionnels.
- 2. Droit à l'effacement (art. 17 RGPD).** Les factures sont **exclues** du droit à l'effacement au titre de l'article 17(3)(b) : obligation comptable de conservation 10 ans. Les autres données sont effaçables à la demande.
- 3. Chiffrement en transit.** Toutes les communications entre le client et le serveur passent par HTTPS (TLS 1.3) via Traefik + Let's Encrypt. L'en-tête `Strict-Transport-Security` force le HTTPS.
- 4. Cloisonnement des données client.** Un client du portail ne peut jamais voir les données d'un autre client du même freelance. L'isolation est garantie par le filtre `entreprise_id` sur chaque requête Doctrine.
- 5. Journalisation des accès aux documents.** Les consultations de documents par les clients sont loggées (`client_id`, `document_id`, `accessed_at`) pour permettre au freelance de prouver la réception en cas de litige.
- 6. Paiement.** TrackR ne stocke jamais les données bancaires. Stripe est le responsable de traitement pour les données de carte. TrackR ne stocke que le `stripe_payment_id` pour la réconciliation.

SECURITE ET ISOLATION

Chaque `Entreprise` est un tenant isolé. Chaque requête authentifiée vérifie que la ressource demandée appartient bien à l'entreprise de l'utilisateur connecté. Un freelance ne peut jamais accéder aux données d'une autre entreprise, un client ne peut jamais voir les données d'un autre client du même freelance.

Au-delà de l'isolation par tenant : mots de passe hachés en **argon2id**, tokens CSRF sur les formulaires publics, vérification MIME et taille sur les uploads, headers de sécurité HTTP (CSP, HSTS, X-Frame-Options) dans Nginx. Les noms de fichiers sont remplacés par un UUID à l'upload pour éviter tout risque de traversée de répertoire.



HEADERS DE SECURITE HTTP

Les headers de securite HTTP sont configures dans `nginx.prod.conf` et appliques a toutes les reponses, qu'elles proviennent de l'API PHP-FPM ou des fichiers statiques React.

Header	Valeur / Description	Protege contre
Strict-Transport-Security	<code>max-age=31536000; includeSubDomains</code>	Downgrade HTTPS
X-Content-Type-Options	<code>nosniff</code> : empeche le type sniffing	MIME confusion
X-Frame-Options	<code>DENY</code> : interdit l'inclusion en iframe	Clickjacking
Content-Security-Policy	<code>default-src 'self'; script-src 'self'</code>	XSS, injection
Referrer-Policy	<code>strict-origin-when-cross-origin</code>	Fuite URL en referer
Permissions-Policy	<code>camera=(), microphone=(), geolocation=()</code>	Acces periphes
X-XSS-Protection	<code>1; mode=block</code> (legacy IE)	XSS legacy

Validation des uploads. Les fichiers uploades par les freelances (logos, documents PDF) sont valides en deux etapes : verification du type MIME via `info_file()` (independant de l'extension) et verification de la taille (2 Mo logo, 10 Mo document). Les fichiers sont stockes avec un UUID comme nom, eliminant les risques de traversee de repertoire (path traversal).



EXTRAITS DE CODE : BACKEND

AuthController::register() : inscription freelance

```
#[Route('/register', name: 'register', methods: ['POST'])]
public function register(
    Request $request,
    UserPasswordHasherInterface $hasher,
    EntityManagerInterface $em,
    UserRepositoryInterface $userRepo,
): JsonResponse {
    $data = json_decode($request->getContent(), true) ?? [];
    $email = trim((string) ($data['email'] ?? ''));
    $password = (string) ($data['password'] ?? '');
    $nom = trim((string) ($data['nom_entreprise'] ?? ''));

    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        return $this->json(['error' => 'Email invalide'], 422);
    }
    if (strlen($password) < 8) {
        return $this->json(['error' => 'Mot de passe trop court (8 min)'], 422);
    }
    if ($userRepo->findByEmail($email) !== null) {
        return $this->json(['error' => 'Email deja utilise'], 409);
    }

    $entreprise = new Entreprise($nom);
    $user = new User($email, $entreprise);
    $user->setPasswordHash($hasher->hashPassword($user, $password));

    $em->persist($entreprise);
    $em->persist($user);
    $em->flush(); // transaction atomique : Entreprise + User ensemble

    return $this->json(['message' => 'verification_sent'], 201);
}
```

La validation est manuelle pour garder les messages d'erreur explicites en francais. Entreprise et User sont persistes dans la meme transaction : si l'un echoue, les deux sont annules. L'utilisation de UserPasswordHasherInterface permet a Symfony de choisir l'algorithme configure (argon2id) sans coupler le code au hasher spécifique.

FactureService::buildSnapshot() : snapshot immuable

```
public function buildSnapshot(Client $client, Entreprise $entreprise): array
{
    return [
        'client' => [
            'nom' => $client->getNom(),
            'email' => $client->getEmail(),
            'adresse' => $client->getAdresse(),
            'ville' => $client->getVille(),
            'pays' => $client->getPays(),
        ],
        'freelance' => [
            'nom' => $entreprise->getNom(),
            'siret' => $entreprise->getSiret(),
            'tva' => $entreprise->getTva(),
            'adresse' => $entreprise->getAdresse(),
            'forme_juridique' => $entreprise->getFormeJuridique(),
            'iban' => $entreprise->getIban(),
            'bic' => $entreprise->getBic(),
        ],
    ];
}
```

Ce tableau est serialise en JSONB et stocke sur Facture.snapshot_client au moment de l'envoi. Il n'est jamais mis a jour : si le client change d'adresse apres emission, la facture conserve les coordonnees initiales, exigence de l'article 289 CGI et de la loi anti-fraude 2018.



StripeController : Signature HMAC du state OAuth

```
private function signState(string $entrepriseId): string
{
    $secret = $_ENV['APP_SECRET'] ?? 'dev';
    $ts     = (string) time();
    $data   = $entrepriseId . '|' . $ts;
    $hmac   = hash_hmac('sha256', $data, $secret);
    return base64_encode($data . '|' . $hmac);
}

private function verifyState(?string $state): ?string
{
    if ($state === null) { return null; }
    $decoded = base64_decode($state, true);
    if ($decoded === false) { return null; }
    $parts = explode('|', $decoded, 3);
    if (count($parts) !== 3) { return null; }
    [$entrepriseId, $ts, $hmac] = $parts;
    if (time() - (int) $ts > 3600) { return null; } // expire apres 1h
    $expected = hash_hmac('sha256', $entrepriseId . '|' . $ts,
        $_ENV['APP_SECRET'] ?? 'dev');
    return hash_equals($expected, $hmac) ? $entrepriseId : null;
}
```

Le state OAuth contient entreprise_id | timestamp | HMAC-SHA256. hash_equals() (comparaison en temps constant) evite les attaques par timing. Un state falsifie, expire ou mal forme est rejete et l'utilisateur est redirige vers /stripe/erreur?raison=state_invalide.



ENDPOINTS API : REFERENCE

Methode	Route	Description
Auth		
POST	/api/auth/register	Inscription freelance (Entreprise + User)
POST	/api/auth/login	Connexion email/password, retourne JWT
GET	/api/me	Profil du freelance connecte
Clients		
GET	/api/clients	Liste des clients de l'entreprise
POST	/api/clients	Creer un client
GET	/api/clients/{id}	Fiche client (isolation tenant)
PUT	/api/clients/{id}	Modifier un client
DELETE	/api/clients/{id}	Supprimer (archive)
POST	/api/clients/{id}/inviter	Generer et envoyer magic-link
PATCH	/api/clients/{id}/accus/activer	Passage prospect vers actif
Projets		
GET	/api/projets	Liste (filtrable par statut)
POST	/api/projets	Creer un projet
GET	/api/projets/{id}	Detail projet + colonnes + taches
PUT	/api/projets/{id}	Modifier un projet
PATCH	/api/projets/{id}/archiver	Transition vers archive
Factures		
GET	/api/factures	Liste avec filtres (statut, client, annee)
POST	/api/factures	Creer en brouillon
GET	/api/factures/{id}	Detail + lignes + snapshot_client
PUT	/api/factures/{id}	Modifier si statut brouillon uniquement
POST	/api/factures/{id}/envoyer	Snapshot, numerotation, PDF, email client + URL Stripe
GET	/api/factures/{id}/pdf	Telecharger le PDF DomPDF
Devis		
GET	/api/devis	Liste des devis
POST	/api/devis	Creer un devis en brouillon
PUT	/api/devis/{id}	Modifier si brouillon
POST	/api/devis/{id}/envoyer	Envoyer par email
PATCH	/api/devis/{id}/signer	Marquer comme signe (apres validation client)
Kanban		
POST	/api/projets/{id}/colonnes	Creer une colonne
PUT	/api/projets/{projetId}/colonnes/{colonneId}	Renommer / reordonner
POST	/api/projets/{id}/taches	Creer une tache
PUT	/api/taches/{id}	Modifier titre / position / colonne
Branding		
GET	/api/branding	Configuration branding
PUT	/api/branding	Logo, couleur, CNAME, nom portail
Stripe		
GET	/api/stripe/config	Statut Stripe Connect
GET	/api/stripe/connect/authorize	URL OAuth Stripe
DELETE	/api/stripe/connect/disconnect	Deconnexion Stripe
POST	/api/stripe/webhook	Webhook Stripe (signature verifiee)
Portail client		
GET	/api/portail/auth/{token}	Echange magic-link contre session_token
GET	/api/portail/moi	Infos ClientAcces (nom, statut)
GET	/api/portail/projets	Projets du client
GET	/api/portail/documents	Documents filtres prospect/actif
GET	/api/portail/factures	Factures du client



06 / DEVELOPPEMENT FRONTEND

React 19 + TypeScript

STACK ET CHOIX TECHNIQUES

React 19	Framework UI. Hooks + Zustand (stores auth, portail, timer). React Query pour le cache serveur. Pas de Redux (scope suffisant).
TypeScript	Typage strict sur toutes les entites API, les props composants et les retours de hooks.
ShadCN/UI	Composants accessibles (ARIA) copies dans le projet et personnalisés. Card, Table, Badge, Dialog, Sheet, Tabs.
Tailwind v3	Utility-first. CSS minimal en production. Palette custom configurée dans <code>tailwind.config.js</code> .
React Router 7	Navigation SPA. Routes protégées (RequireAuth), routes publiques portail (Require-Client).
Vite 8	Build tool et serveur de développement HMR. Compatible Vitest 4 pour les tests unitaires.

DEUX INTERFACES DISTINCTES

TrackR expose deux interfaces qui n'ont rien en commun visuellement ni techniquement :

Dashboard freelance : interface dense orientée gestion. Sidebar de navigation permanente, vues tabulaires pour les clients/factures/projets, formulaires de création et d'édition, Kanban drag-and-drop par projet. L'accent visuel utilise la couleur primaire de l'application (teal 1A9E8A), les composants sont issus de ShadCN/UI.

Portail client : interface épurée orientée action. Accessible par magic-link uniquement (pas de mot de passe). Le branding est entièrement configurable (logo, couleur primaire, nom du portail). Le client voit : accueil (résumé du projet), documents, Kanban en lecture, factures à payer. L'interface ne mentionne jamais le nom "TrackR".

DESIGN SYSTEM ET THEMING WHITE-LABEL

Le dashboard freelance a une palette fixe. Le portail client, lui, doit changer de couleur et de logo selon le freelance qui l'utilise. Ces deux besoins ont amené à deux approches différentes dans le même codebase.

Dashboard freelance. La palette est statique : fond crème (FAF7F7), accent teal (1A9E8A), texte encre (16140F). Elle est définie dans `index.css` via des variables CSS personnalisées (`--accent`, `--paper`, `--ink`) et appliquée via les classes Tailwind et les composants ShadCN.

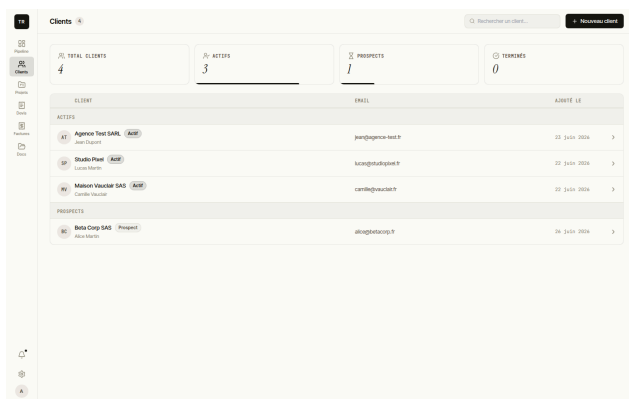


Portail client (white-label). La couleur primaire et le logo sont charges depuis l'API au moment de l'accès (GET /api/portail/branding). La couleur est injectee en variable CSS sur :root par le hook useBranding() :

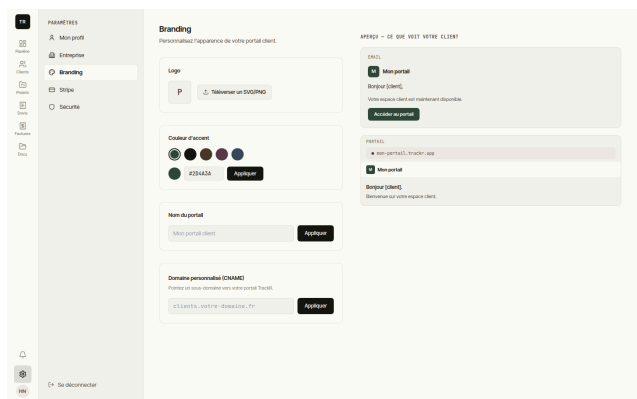
```
function useBranding() {
  const { data } = useQuery({ queryKey: ['branding'], queryFn: fetchBranding })
  useEffect(() => {
    if (data?.couleur_primaire) {
      document.documentElement.style.setProperty(
        '--color-primary', data.couleur_primaire
      )
    }
  }, [data?.couleur_primaire])
  return data
}
```

Tous les composants ShadCN du portail utilisent `var(--color-primary)` comme couleur d'accent. Changer la couleur primaire dans les parametres freelance est repercute instantanement dans le portail client, sans redeploiement ni rebuild CSS.

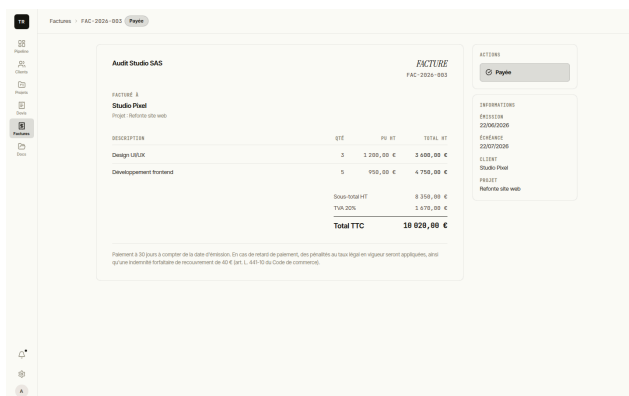
CAPTURES D'ECRAN



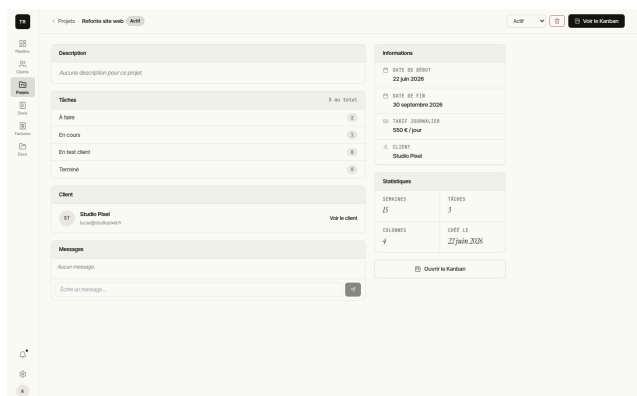
Gestion des clients : liste et statuts



Configuration branding : aperçu en direct



Détail facture : statut et paiement Stripe



Détail projet : Kanban et suivi



ARCHITECTURE FRONTEND : ROUTING ET GARDE-FOUS

App.tsx (React Router 7) centralise toutes les routes et separe les deux interfaces en deux groupes de routes gardees independants.

```
function App() {
  return (
    <Router>
      <Routes>
        { /* Routes publiques */ }
        <Route path="/login" element={<LoginPage />} />
        <Route path="/register" element={<RegisterPage />} />
        <Route path="/portail/acces" element={<PortailAccesPage />} />

        { /* Dashboard freelance - necessite JWT valide (PrivateRoute) */ }
        <Route element={<PrivateRoute />>
          <Route path="/" element={<DashboardPage />} />
          <Route path="/clients" element={<ClientsPage />} />
          <Route path="/clients/:id" element={<ClientDetailPage />} />
          <Route path="/projets" element={<ProjetsPage />} />
          <Route path="/projets/:id/kanban" element={<KanbanPage />} />
          <Route path="/factures" element={<FacturesPage />} />
          <Route path="/devis" element={<DevisPage />} />
          <Route path="/documents" element={<DocumentsPage />} />
          <Route path="/parametres" element={<ParametresLayout />} />
          <Route index element={<ProfilPage />} />
          <Route path="branding" element={<BrandingPage />} />
          <Route path="stripe" element={<StripePage />} />
          <Route path="securite" element={<SecuritePage />} />
        </Route>
      </Routes>

      { /* Portail client - necessite session_token cookie (PortailRoute) */ }
      <Route element={<PortailRoute />>
        <Route path="/portail" element={<PortailAccueil />} />
        <Route path="/portail/projets" element={<PortailProjets />} />
        <Route path="/portail/factures" element={<PortailFactures />} />
        <Route path="/portail/documents" element={<PortailDocuments />} />
        <Route path="/portail/kanban" element={<PortailKanban />} />
      </Route>
    </Routes>
  </Router>
)
}
```

PrivateRoute verifie la presence d'un JWT valide via useAuth() et redirige vers /login si absent. PortailRoute verifie le cookie session_token et redirige vers /portail/acces si absent. Les deux garde-fous sont totalement independants : le dashboard freelance et le portail client sont deux SPA qui partagent seulement les types TypeScript de l'API.

KANBAN : DRAG-AND-DROP ET PERSISTANCE

Le Kanban est la fonctionnalite visuellement la plus marquante du dashboard. Le drag-and-drop est implemente avec l'API HTML5 native (draggable, onDragStart, onDrop) sans dependance externe. Ce choix evite un bundle supplementaire et reste suffisant pour les besoins du projet (colonnes verticales, pas de tri croise complexe).

Persistance des positions. Le deplacement d'une tache declenche un appel PUT /api/taches/{id} avec le nouveau colonne_id et la nouvelle position (entier). Les positions sont recalculees cote serveur apres chaque deplacement pour eviter les conflits d'ordre apres plusieurs deplacements consecutifs.

```
// Tache draggable -- API HTML5 native
<div
  draggable
  onMouseDown={() => { wasDragged.current = false }}
>
```



```

onDragStart={e) => {
  wasDragged.current = true
  e.dataTransfer.setData('tacheId', tache.id)
  e.dataTransfer.setData('fromColonneId', colonne.id)
}}
>
{/* contenu de la tache */}
</div>

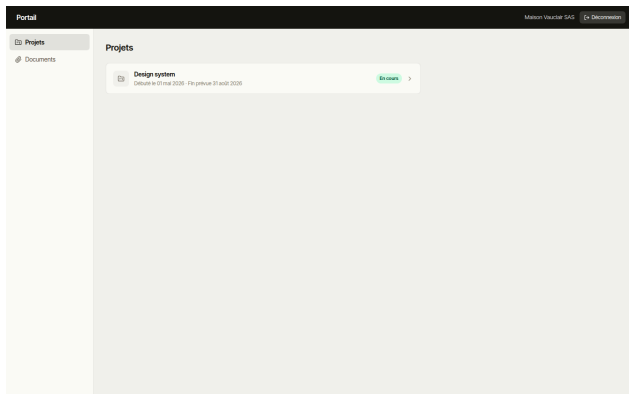
// Zone de depot sur la colonne
<div>
  onDragOver={e) => e.preventDefault()}
  onDrop={e) => {
    const tacheId = e.dataTransfer.getData('tacheId')
    const fromColonneId = e.dataTransfer.getData('fromColonneId')
    if (tacheId && fromColonneId !== colonne.id) {
      mutateTache({ id: tacheId, colonne_id: colonne.id })
    }
  }}
  >

```

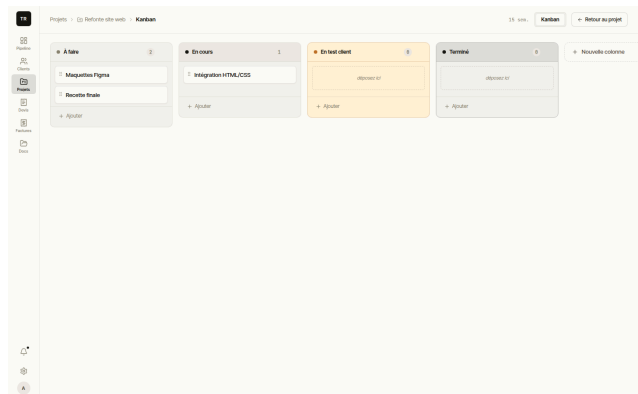
`mutateTache` utilise une mutation React Query : le déplacement persiste cote serveur via PUT `/api/taches/{id}`. La vue portail client affiche le Kanban en lecture seule (colonnes `visible_client = true` uniquement) sans interaction drag.

CAPTURES D'ECRAN : PORTAIL CLIENT WHITE-LABEL

Le portail client est entièrement distinct du dashboard. Le branding (logo, couleur primaire, nom du portail) est chargé depuis l'API au premier accès et applique dynamiquement via des variables CSS. Le client ne voit jamais le nom "TrackR".



Portail client : accueil avec branding freelance



Kanban partage : colonnes visibles par le client



07 / TESTS ET QUALITE

Strategie de test

APPROCHE

Les tests couvrent deux niveaux : tests fonctionnels backend avec PHPUnit (sur une vraie base PostgreSQL, pas un mock) et tests unitaires frontend avec Vitest. Ils s'exécutent automatiquement à chaque push dans le pipeline CI/CD. Des scénarios Behat étaient prévus pour couvrir les parcours complets, mais ont été écartés face au calendrier ; ces parcours ont été validés manuellement via Postman et navigation directe.

TESTS BACKEND : PHPUNIT

Les tests backend utilisent **PHPUnit avec WebTestCase** (tests fonctionnels qui envoient de vraies requêtes HTTP au kernel Symfony). Chaque test utilise une base de données PostgreSQL réelle (pas de mocks) avec des fixtures injectées avant chaque test.

Scénarios couverts : inscription/connexion (JWT), création et envoi de facture (numérotation séquentielle, statut machine à états), flux magic-link (création token, expiration, usage unique), isolation multi-tenant (un freelance ne peut pas accéder aux données d'un autre), webhooks Stripe (paiement reçu, mise à jour statut).

VALIDATION DES PARCOURS UTILISATEURS

Les parcours utilisateurs critiques ont été validés manuellement tout au long du développement. Behat (framework BDD Symfony) avait été identifié comme l'outil cible pour automatiser ces parcours : les scénarios suivants avaient été spécifiés mais non implémentés faute de temps :

1. Inscription freelance, configuration branding, création d'un client, invitation portail.
2. Accès portail client via magic-link, visualisation des documents, paiement facture (mode test Stripe).
3. Création devis, envoi au client, conversion en facture, envoi et relance automatique.
4. Kanban : création colonne, déplacement tâche, validation par le client.

PIPELINE CI : GITHUB ACTIONS

Le pipeline CI comporte **4 jobs** enchaînés :



frontend	ESLint, TypeScript typecheck, Vitest (tests unitaires), Vite build.
backend	PHPUnit avec PostgreSQL 17 et Redis 7.4 reels (pas de mocks). En parallele de frontend.
publish	Build deux images Docker (trackr-api + trackr-frontend), push sur ghcr.io. Conditionnel : main uniquement.
deploy	SSH Hetzner : <code>docker compose pull && up -d</code> , prune images obsoletes.



PIPELINE CI : EXTRAIT GITHUB ACTIONS

```

name: CI/CD
on:
  push:
    branches: [main]

jobs:
  frontend:
    # lint → typecheck → Vitest → build Vite
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '22', cache: 'npm' }
      - run: npm ci && npm run lint && npm run typecheck
        working-directory: frontend
      - run: npm run test -- --run && npm run build
        working-directory: frontend

  backend:
    # composer → migrations → PHPUnit (postgres:17 + redis:7.4 reels)
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:17-alpine }
      redis: { image: redis:7.4-alpine }
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.4', extensions: pdo_pgsql,sodium,gd,intl }
      - run: composer install --no-interaction
        working-directory: api
      - run: php bin/console doctrine:migrations:migrate --no-interaction
        working-directory: api
      - run: ./vendor/bin/phpunit
        working-directory: api
        env: { APP_ENV: test, DATABASE_URL: "postgresql://trackr:trackr@localhost/trackr_test" }
      - run: ./vendor/bin/phpunit
        working-directory: api
        env: { APP_ENV: test }

  publish:
    # build trackr-api + trackr-frontend → push ghcr.io
    needs: [frontend, backend]
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/build-push-action@v6
        with: { file: docker/php/Dockerfile, push: true,
          tags: ghcr.io/hamza-naya/trackr-api:latest }
      - uses: docker/build-push-action@v6
        with: { context: frontend, push: true,
          tags: ghcr.io/hamza-naya/trackr-frontend:latest }

  deploy:
    # SSH Hetzner → docker compose pull → up -d → prune
    needs: publish
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${ secrets.SERVER_HOST }
          key: ${ secrets.SERVER_SSH_KEY }
          script: |
            cd /opt/trackr
            docker compose -f docker-compose.prod.yml pull
            docker compose -f docker-compose.prod.yml up -d --remove-orphans
            docker image prune -f

```

Les jobs frontend et backend s'exécutent en parallèle. Le job publish (conditionnel main) construit deux images Docker (trackr-api et trackr-frontend) et les pousse sur GitHub Container Registry. Le job deploy se connecte en SSH au VPS Hetzner et exécute docker compose pull && up. La base PostgreSQL est un vrai service Docker dans le job CI, pas un mock, ce qui garantit que les migrations et les contraintes FK sont testées.



EXEMPLE DE TEST PHPUNIT : ISOLATION MULTI-TENANT

```

class ClientControllerTest extends WebTestCase
{
    private KernelBrowser $client;
    private string $token;

    protected function setUp(): void
    {
        $this->client = static::createClient();
        $em = static::getContainer()->get(EntityManagerInterface::class);
        $em->getConnection()->executeStatement('TRUNCATE entreprise CASCADE');
        static::ensureKernelShutdown();

        // Inscription + recuperation du JWT
        $this->client = static::createClient();
        $this->client->request('POST', '/api/auth/register', [], [],
            ['CONTENT_TYPE' => 'application/json'],
            json_encode(['email' => 'a@test.com', 'password' => 'secret123',
                'nom_entreprise' => 'Tenant A']))
        );
        $this->token = json_decode(
            $this->client->getResponse()->getContent(), true
        )['token'];
    }

    public function testIsolationCrossTenant(): void
    {
        // Creer un client pour Tenant A
        $this->client->request('POST', '/api/clients', [], [],
            ['CONTENT_TYPE' => 'application/json',
                'HTTP_AUTHORIZATION' => 'Bearer ' . $this->token],
            json_encode(['nom' => 'Client de A', 'email' => 'c@a.com']))
        );
        $clientId = json_decode(
            $this->client->getResponse()->getContent(), true
        )['id'];

        // Tenant B essaie d'accéder au client de A -- doit recevoir 404
        static::ensureKernelShutdown();
        $browser2 = static::createClient();
        $browser2->request('POST', '/api/auth/register', [], [],
            ['CONTENT_TYPE' => 'application/json'],
            json_encode(['email' => 'b@test.com', 'password' => 'secret456',
                'nom_entreprise' => 'Tenant B']))
        );
        $tokenB = json_decode(
            $browser2->getResponse()->getContent(), true
        )['token'];

        $browser2->request('GET', '/api/clients/' . $clientId, [], [],
            ['HTTP_AUTHORIZATION' => 'Bearer ' . $tokenB])
        );
        $this->assertResponseStatusCodeSame(404);
    }
}

```

Ce test vérifie l'invariant de sécurité le plus critique : un freelance ne peut jamais accéder aux ressources d'un autre tenant, même en connaissant l'UUID d'une ressource. La réponse est 404 et non 403 pour ne pas confirmer l'existence de la ressource.



COUVERTURE DES TESTS PAR MODULE

Module	Tests PHPUnit	Scenarios couverts
Authentification	6	Register, login, email invalide, mdp court, duplique, mauvais mdp
Clients	7	CRUD, statuts, isolation tenant, UUID inconnu
Portail auth	8	Magic-link, expiration, usage unique, isolation cross-tenant
Portail acces	7	Sessions, filtre prospect vs actif, documents filtres
Documents	8	Upload, MIME invalide, taille, visibilite, suppression, isolation
Facturation	9	Creation, numerotation sans trous, envoi, immuabilite, PDF, webhook
Kanban	6	Colonnes, taches, deplacement, notification "En test client"
Branding	5	CRUD, validation couleur regex, logo upload, suppression
Stripe	4	Config, OAuth URL, webhook signature, disconnect
Total	150+	154 methodes sur 16 fichiers : isolation multi-tenant testee sur chaque module



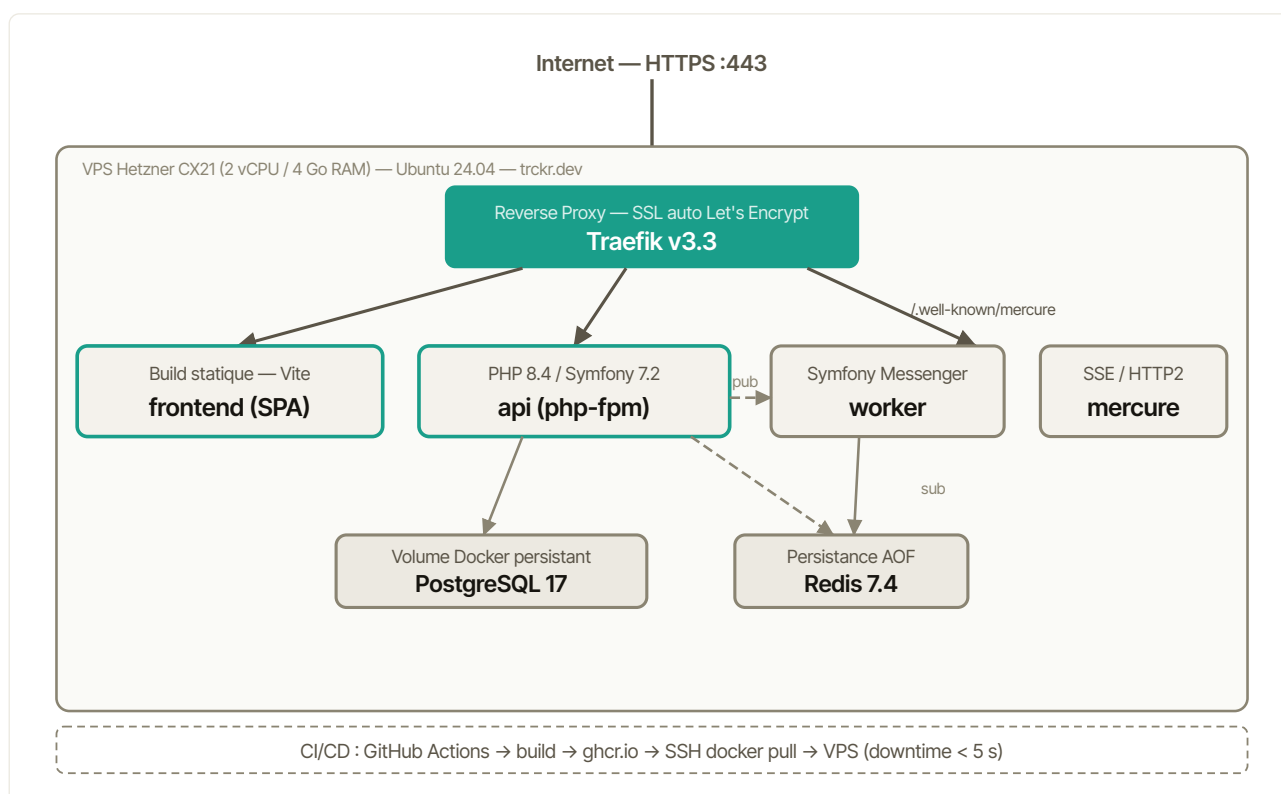
Infrastructure et mise en production

ARCHITECTURE DE DEPLOIEMENT

L'application est deployee sur un **VPS Hetzner** (CX21, 2 vCPU / 4 Go RAM, Ubuntu 24.04) sous le domaine **trckr.dev**. Tous les services tournent dans des conteneurs Docker orchestres par Docker Compose.

traefik	Reverse proxy v3.3. SSL automatique Let's Encrypt. Routing par domaine et prefixe de chemin.
frontend	Nginx 1.27 servant la SPA React (build statique Vite). Proxy <code>/api</code> vers le conteneur api.
api	Nginx + PHP-FPM 8.4 + Symfony 7.2, geres par Supervisor. Image multi-stage Alpine (142 Mo).
worker	Consumer Symfony Messenger (emails async, PDF). Meme image que api.
mercure	Hub Mercure pour les notifications temps reel (SSE). Proxy par Traefik sur <code>/.well-known/mercure</code> .
postgres	PostgreSQL 17. Volume Docker persistant. Backup quotidien via cron + <code>pg_dump</code> .
redis	Redis 7.4. File Messenger et cache. Persistance AOF activee.

SCHEMA D'ARCHITECTURE : SERVICES DOCKER





Vue d'ensemble des 7 services Docker Compose en production. Traefik route vers le conteneur frontend (Nginx + SPA React) qui proxy /api vers l'api (Nginx + PHP-FPM). Le worker partage la meme image que l'api.

DOCKERFILE MULTI-STAGE

Le Dockerfile utilise trois stages pour garder l'image finale la plus legere possible (142 Mo) :

dev	php-fpm-alpine + Composer + toutes les dependances. Utilise uniquement en developpement local.
builder	<code>composer install --no-dev, cache:warmup</code> . Prepare les vendors optimises.
runtime	Image minimale (php-fpm-alpine). Copie uniquement les vendors et le code depuis builder. 142 Mo final.

PIPELINE DE DEPLOIEMENT

A chaque push sur `main`, GitHub Actions construit les deux images Docker et les pousse sur GitHub Container Registry. Ensuite, il se connecte au VPS en SSH et execute `docker compose pull && up` : les nouveaux conteneurs remplacent les anciens, les migrations Doctrine se lancent automatiquement. Les secrets (cles Stripe, JWT, etc.) sont stockes dans GitHub Secrets et ne transitent jamais en clair.



DOCKERFILE : EXTRAIT ANNOTE

```
# Stage 1 : dev (environnement local uniquement)
FROM php:8.4-fpm-alpine AS dev
RUN apk add --no-cache libpq-dev icu-dev oniguruma-dev git \
    && docker-php-ext-install pdo_pgsql intl mbstring opcache
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
WORKDIR /var/www/html

# Stage 2 : builder (install production uniquement)
FROM dev AS builder
COPY api/composer.json api/composer.lock ./
RUN composer install \
    --no-dev \
    --optimize-autoloader \
    --no-scripts
COPY api/ .
ENV APP_ENV=prod \
    APP_SECRET=dummy \
    DATABASE_URL=postgresql://x:x@x/x \
    MAILER_DSN=null://null
RUN bin/console cache:warmup --env=prod

# Stage 3 : runtime (~142 Mo, image minimale)
FROM php:8.4-fpm-alpine AS runtime
RUN apk add --no-cache nginx supervisor libpq icu icu-libs \
    && docker-php-ext-install pdo_pgsql opcache
COPY --from=builder /var/www/html /var/www/html
COPY docker/nginx/default.conf /etc/nginx/http.d/default.conf
COPY docker/supervisord.conf /etc/supervisord.conf
EXPOSE 80
CMD ["/usr/bin/supervisord", "-c", "/etc/supervisord.conf"]
```

Le stage `builder` installe uniquement les dépendances de production (`--no-dev`) et chauffe le cache Symfony avec des variables d'environnement factices. Le stage `runtime` ne copie que le résultat : aucun outil de build (Composer, Git, compilation C) n'est présent dans l'image finale. La séparation dev/builder/runtime est la pratique recommandée pour limiter la surface d'attaque de l'image.

SERVICES DOCKER COMPOSE EN PRODUCTION

```
services:
  traefik:
    image: traefik:v3.3
    ports: ['80:80', '443:443']
    volumes:
      - ./traefik-dynamic.yml:/etc/traefik/dynamic.yml:ro
      - traefik_letsencrypt:/letsencrypt
    command:
      - --providers.file.filename=/etc/traefik/dynamic.yml
      - --entrypoints.websecure.address=:443
      - --certificatesresolvers.le.acme.email=${ACME_EMAIL}
      - --certificatesresolvers.le.acme.storage=/letsencrypt/acme.json

  frontend:
    image: ghcr.io/hamza-naya/trackr-frontend:${TAG:-latest}
    depends_on: [api, mercure]

  api:
    image: ghcr.io/hamza-naya/trackr-api:${TAG:-latest}
    environment:
      APP_ENV: prod
      DATABASE_URL: ${DATABASE_URL}
      STRIPE_SECRET_KEY: ${STRIPE_SECRET_KEY}
      MAILER_DSN: ${MAILER_DSN}
    depends_on: [postgres, redis]

  worker:
    image: ghcr.io/hamza-naya/trackr-api:${TAG:-latest}
    command: php bin/console messenger:consume async --time-limit=3600 -vv
    depends_on: [api, redis]
    restart: unless-stopped

  mercure:
    image: dunglas/mercure
    environment:
```



```
SERVER_NAME: ':80'
MERCURE_PUBLISHER_JWT_KEY: ${MERCURE_JWT_SECRET}

postgres:
  image: postgres:17-alpine
  volumes: ['postgres_data:/var/lib/postgresql/data']
  environment:
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}

redis:
  image: redis:7.4-alpine
  command: redis-server --appendonly yes
```

Le worker utilise la meme image que api avec une commande differente (`messenger:consume`). Ce pattern evite de maintenir deux Dockerfiles distincts pour le meme code PHP. `--time-limit=3600` provoque un arret propre toutes les heures que Docker (`restart: unless-stopped`) gere en relancant automatiquement le conteneur.



Recherches et comparaisons techniques

MERCURE VS WEBSOCKETS : NOTIFICATIONS TEMPS REEL

Pour les notifications in-app (paiement reçu, nouveau commentaire), deux approches ont été comparées : **WebSockets** (connexion TCP bidirectionnelle persistante) et **Mercure** (protocole SSE basé sur HTTP/2, supporté nativement par Symfony).

J'ai choisi Mercure. Le hub est un seul binaire Go, pas de serveur Node à gérer en parallèle. Il s'intègre nativement avec Symfony via le bundle officiel, et Traefik le route comme n'importe quelle requête HTTP. Avec WebSockets, il aurait fallu un serveur dédié (Ratchet ou Swoole) et adapter toute la configuration Traefik pour les connexions persistantes.

STRIPE CONNECT STANDARD VS EXPRESS

Stripe Connect propose deux modes : **Standard** (le freelance crée ou connecte son propre compte Stripe existant via OAuth) et **Express** (Stripe crée un compte simplifié au nom du freelance, moins de friction à l'onboarding).

J'ai retenu le mode **Standard** : les freelances cibles ont souvent déjà un compte Stripe. Il offre plus de contrôle sur le compte (le freelance voit ses transactions directement dans son dashboard Stripe) et n'implique pas de frais supplémentaires Stripe par transaction. Le mode Express aurait été plus adapté si TrackR avait voulu prélever une commission (via `application_fee_amount`), ce qui n'est pas le cas.

SHADCN/UI VS ALTERNATIVES

Pour la librairie de composants frontend, j'ai comparé **ShadCN/UI**, **MUI (Material UI)** et **Radix UI** seul. J'ai retenu ShadCN/UI parce que les composants sont copiés dans le projet (pas de dépendance npm externe à maintenir), pleinement personnalisables via Tailwind, accessibles (base Radix UI), et bien documentés.

MUI impose un thème Material Design difficile à désactiver et un overhead de bundle significatif. Radix UI seul aurait nécessité de styliser chaque composant manuellement.



POSTGRESQL JSONB : CAS DU SNAPSHOT CLIENT

Le champ `snapshot_client` sur la table `facture` stocke un document JSON contenant les coordonnées du client au moment de l'émission (nom, adresse, SIRET, statut TVA). Si le client change d'adresse après, la facture garde les informations d'origine, comme l'exige la loi anti-fraude 2018.

J'ai choisi une colonne JSONB plutôt qu'une table de snapshot séparée : ce document n'est jamais requête par champ interne, seulement lu en entier. Une table séparée aurait été plus propre relationnellement mais plus complexe sans avantage réel ici.

ARGON2ID VS BCrypt : HASHAGE DES MOTS DE PASSE

Symfony propose `argon2id` par défaut depuis la version 6.3. Avant de l'adopter, j'ai comparé avec `bcrypt`, qui reste la référence dans beaucoup de projets plus anciens.

Argon2id a été retenu :

- 1. Resistance memoire.** Argon2id est conçu pour être coûteux en RAM (paramètre `memory_cost`, par défaut 64 Mo dans Symfony). Les attaques par GPU parallèle sont freinées car chaque tentative nécessite un bloc mémoire important, contrainte que `bcrypt` n'impose pas.
- 2. Double resistance.** Argon2id combine les propriétés d'Argon2d (résistant aux attaques GPU) et d'Argon2i (résistant aux attaques par canal auxiliaire via accès mémoire indépendant des données). `bcrypt` ne protège que contre la force brute classique.
- 3. Support natif Symfony 7.** `UserPasswordHasherInterface` utilise `argon2id` par défaut depuis Symfony 6.3. Aucune configuration supplémentaire dans `security.yaml`.
- 4. OWASP Password Storage Cheat Sheet (2023).** Argon2id est classé en premier, avant `scrypt` et `bcrypt`. La recommandation est de l'utiliser avec `memory=19 Mo`, `iterations=2`, `parallelism=1` au minimum.

En pratique, le hashage `argon2id` n'est appelé qu'à l'inscription et à la connexion. Le surcoût en temps CPU (environ 50 ms en local) est négligeable pour une SaaS et contribue directement à la sécurité des comptes freelance.



API PLATFORM VS CONTROLLERS MANUELS

L'écosystème Symfony propose **API Platform** comme solution standard pour la construction d'APIs REST. TrackR utilise à la place des contrôleurs Symfony manuels. Ce choix a été documenté dès le Sprint 0.

Ce qu'API Platform aurait apporté. Génération automatique des routes CRUD, sérialisation via JMS/Symfony Serializer, documentation OpenAPI depuis les attributs Doctrine, filtres et pagination natifs, support JSON:API et JSON-LD.

Pourquoi les contrôleurs manuels ont été préférés. Dans le contexte d'un projet de certification orale, chaque étape du flux HTTP doit pouvoir être expliquée précisément. Avec API Platform, la route `/api/clients` est générée par introspection. Expliquer POURQUOI elle répond ce qu'elle répond nécessite de connaître les internals du bundle. Avec un contrôleur manuel, le flux est lisible dans le code : désérialisation manuelle, validation, appel repository, sérialisation `toArray()`, retour JSON. Chaque étape est explicable et justifiable.

Inconvénients gérés. Pagination manuelle avec `LimitOffsetPaginator` Doctrine. Documentation via `NelmioApiDocBundle 4.x` avec attributs PHP 8. Validation avec messages d'erreur explicites en français.

Bilan. C'est un compromis entre lisibilité du code et productivité de développement. Dans un contexte d'équipe avec une codebase à maintenir sur plusieurs années, API Platform serait probablement le bon choix. Pour un projet de certification où chaque étape du flux doit être expliquée à l'oral, les contrôleurs manuels ont l'avantage d'être entièrement transparents.



10 / BILAN ET PERSPECTIVES

Ce que ce projet m'a apporté

COMPETENCES ACQUISES

En solo, il n'y a personne pour valider tes choix à ta place. Ça oblige à réfléchir vraiment à chaque décision : pourquoi cette architecture, pourquoi cette table, pourquoi ce comportement d'authentification. C'est ce que j'ai trouvé le plus formateur dans ce projet.

Ce que je retiens concrètement : la modélisation Merise sur un vrai schéma PostgreSQL (pas un exercice sur papier), le double firewall Symfony pour deux audiences avec des mécanismes d'auth différents, et le flux OAuth Stripe avec sécurisation HMAC. Trois sujets que je peux expliquer en détail parce que j'y ai passé du temps et que j'ai dû comprendre pourquoi ça ne fonctionnait pas avant de le faire marcher.

DIFFICULTES RENCONTREES

Gestion du périmètre solo : conduire un projet full-stack complet seul sur 8 mois nécessite une discipline forte sur les priorités. Plusieurs fonctionnalités ont été délibérément écartées du MVP (e-signature native, chat temps réel, app mobile) pour tenir les délais.

Conformité légale de la facturation : les 16 mentions obligatoires de l'article 289 CGI et les exigences de la loi anti-fraude 2018 (immutabilité, numérotation sans trous) ont nécessité une recherche approfondie et des décisions de conception spécifiques (`snapshot_client`, séquences PostgreSQL).

Infrastructure CI/CD : la mise en place du pipeline a été itérative : chaque correction de build révélait une nouvelle erreur (conflits Flex, headers de compilation manquants dans le stage Docker runtime, configuration du worker Messenger). Ces problèmes, résolus en Sprint 0, ont ensuite permis de travailler sereinement.

Double firewall Symfony : la coexistence de deux mécanismes d'authentification (JWT pour les freelances, `session_token` pour les clients) dans la même application Symfony a nécessité une compréhension approfondie du composant Security. Le firewall `portail` et le firewall `api` doivent être déclarés dans le bon ordre dans `security.yaml` pour que les patterns de routes ne se chevauchent pas.



CORRECTIONS ET STABILISATION v1

Le dernier sprint (S18) a été entièrement consacré à la stabilisation de la v1 avant soutenance. Il ne s'agissait pas de nouvelles fonctionnalités mais de corrections et d'ajustements identifiés pendant les phases de test.

Refonte de l'authentification portail. Le mécanisme de refresh du `session_token` client ne gère pas correctement l'expiration en milieu de session. Le client se retrouvait déconnecté sans message explicite. Correction : vérification proactive de l'expiration au chargement de chaque page protégée et redirection vers `/portail/acces` avec un message clair.

Conformité légale de la facturation. Relecture complète des 16 mentions CGI sur un PDF généré en conditions réelles. Deux mentions manquaient : la période de réalisation de la prestation (mention 3) et les conditions de paiement complètes (mention 15). Ajoutées dans le template DomPDF.

Corrections UX. Plusieurs comportements inattendus identifiés : le bouton "Envoyer la facture" restait actif sur une facture déjà envoyée, le portail client n'affichait pas de message d'erreur en cas de lien expiré (page blanche), et le Kanban ne persistait pas l'ordre des colonnes après rechargement. Ces trois points ont été corrigés en S18.

Nettoyage des variables d'environnement. Deux variables référencées dans le code n'étaient pas documentées dans `.env.example`, ce qui aurait rendu le déploiement impossible sans debug. Ajout d'un `.env.example` complet avec valeurs d'exemple.

CE QUE JE FERAIS DIFFÉREMMENT

- 1. Tests de parcours utilisateurs plus tôt.** L'absence de tests E2E automatisés (Behat) a allongé le temps de détection des bugs d'intégration, notamment les redirections post-paiement Stripe. Ces tests auraient dû être écrits en parallèle du développement des features, pas en fin de projet.
- 2. API Platform pour les CRUD standards.** Avec du recul, les endpoints CRUD (clients, projets, tâches) auraient pu utiliser API Platform avec des contrôleurs custom pour les endpoints métier (envoi facture, webhook). Le mix aurait été plus pragmatique.
- 3. Variables d'environnement typées.** Les variables Symfony (`DATABASE_URL`, `STRIPE_SECRET_KEY`) sont des strings sans validation. Un fichier `.env.schema` ou un service de configuration type-safe aurait évité plusieurs bugs de déploiement (variable manquante détectée à l'exécution et non au démarrage).
- 4. Gestion des fichiers avec un service dédié.** Les fichiers PDF et logos sont stockés dans `var/uploads/` sur le même serveur que l'application. En production, un service objet comme S3 ou Scaleway Object Storage serait plus robuste (réplication, CDN, séparation des préoccupations).



ROADMAP v2 : FONCTIONNALITES ENVISAGEES

1. **E-signature** (YouSign API) : integration d'un prestataire qualifie eIDAS pour la signature electronique des contrats et devis directement dans le portail.
2. **Notifications push mobile** : integration d'un service de push natif (FCM/APNs) pour alerter le freelance sur smartphone sans passer par l'application web.
3. **Domaine custom** : verification CNAME automatique par polling DNS pour que le portail soit accessible depuis le domaine du freelance.
4. **Stripe Billing** : abonnement mensuel TrackR directement depuis l'interface, avec gestion des plans et des periodes d'essai.



COUVERTURE DES COMPETENCES RNCP37873

Le titre RNCP37873 (Concepteur Developpeur d'Applications) couvre 11 competences reparties en 3 blocs CCP. Le tableau suivant met en correspondance chaque competence avec les livrables concrets du projet TrackR.

CCP	Competence	Livrable TrackR
1.1	Maquetter une application	19 maquettes (Figma), design system ShadCN/UI, 2 interfaces distinctes (dashboard freelance + portail client)
1.2	Realiser une interface utilisateur web statique et adaptable	Layout Tailwind v3 responsive, shell sidebar + header, CSS variables pour le branding client
1.3	Developper une interface utilisateur web dynamique	React 19, hooks custom (useAuth, useBranding), drag-and-drop Kanban (HTML5 natif), SPA React Router 7
1.4	Contribuer a la gestion d'un projet informatique	18 sprints documentes, journal de bord technique sprint par sprint, 25 US MoSCoW, Conventional Commits verifiables sur GitHub
2.1	Concevoir et mettre en oeuvre une base de donnees	Merise MCD vers MLD vers DDL PostgreSQL 17, 16 entites, normalisation 3FN, 12 index
2.2	Developper des composants dans le langage d'une base de donnees	Sequences PostgreSQL (numeration facture), JSONB (snapshot_client), index partiels conditionnes
2.3	Developper des composants d'accès aux donnees	14 Repositories Doctrine (findByEntreprise, findByToken...), isolation multi-tenant, migrations SQL pures
3.1	Collaborer a la gestion d'un projet informatique et interagir avec les intervenants	Conventional Commits (feat:, fix:, chore:, ci:), historique verifiable sur GitHub, cahier des charges (25 US, criteres d'acceptation)
3.2	Concevoir et developper des applications multicouches reparties	Architecture Controller / Service / Repository / Entity, Symfony 7.2, Messenger async, double firewall
3.3	Preparer et executer les plans de tests et de validation	150+ tests PHPUnit WebTestCase (154 methodes / 16 fichiers) + 23 tests Vitest frontend, validation manuelle Postman, PostgreSQL et Redis reels en CI
3.4	Preparer et executer le deploiement d'une application	Docker multi-stage (142 Mo), GitHub Actions CI/CD, Hetzner VPS Ubuntu 24.04, Traefik v3.3 SSL automatique

SYNTHESE

TrackR couvre l'integralite des 11 competences RNCP37873 avec des livrables concrets et verifiables. Les competences 2.2 (composants BDD) et 3.2 (architecture multicouche) sont celles qui ont necessite le plus de recherches : conformite legale de la facturation (art. 289 CGI, loi anti-fraude 2018), securite OAuth (HMAC state), double firewall Symfony (JWT freelance + session_token client). Ces problemes techniques, resolus sprint par sprint, constituent la base des exemples de pratique professionnelle du dossier.



DECISIONS D'ARCHITECTURE : RECAPITULATIF

Le tableau suivant recapitule les principales decisions d'architecture de TrackR, la solution retenue et l'alternative ecartee avec sa justification.

Sujet	Choix retenu	Alternative ecartee
Backend	Symfony 7.2, controllers manuels	API Platform (abstraction opaque pour oral)
ORM	Doctrine ORM 3 + migrations SQL pures	Eloquent Laravel (ecosysteme different)
Base de donnees	PostgreSQL 17 (UUID natif, JSONB, sequences)	MySQL 8 (types avances moins etendus)
File messages	Redis 7.4 + Symfony Messenger	RabbitMQ (complexite infra injustifiee)
Auth freelance	JWT stateless (LexikJWT, RS256)	Sessions PHP (stateful, probleme multi-instances)
Auth client	Magic-link + session_token cookie httpOnly	OAuth Google (dependance tierce, friction)
Paieement	Stripe Connect Standard (fonds directs)	PayPal (taux conversion B2B FR moindre)
Notif temps reel	Mercure SSE/HTTP2	WebSockets Ratchet (serveur de die requis)
Framework UI	React 19 + ShadCN/UI + Tailwind v3	Vue 3 (ecosysteme ShadCN moins mature)
Build tool	Vite 8 + Vitest 4	Create React App (archive), Webpack (plus lent)
Reverse proxy	Traefik v3.3 (SSL auto Let's Encrypt)	Nginx + Certbot (renouvellement manuel)
Hashage mdp	Argon2id (OWASP 2023)	bcrypt (moins resistant GPU)
Generation PDF	DomPDF (HTML vers PDF, Twig templates)	Gotenberg (service Docker supplementaire)
Stockage uploads	var/uploads/ sur VPS (MVP)	S3 Scaleway (v1.1 apres stabilisation)

PRINCIPE DIRECTEUR

Les decisions de TrackR privilegient l'**explicabilite et la maintenabilite solo** sur la productivite pure. Dans un projet en equipe, plusieurs choix seraient differents : API Platform pour les CRUD, RabbitMQ pour la fiabilite des messages, S3 pour le stockage. Ces arbitrages sont documentes et justifiables precisement, ce qui est l'objectif principal dans le cadre d'une certification orale.



GLOSSAIRE

Termes et acronymes

API (REST)	Interface de programmation exposant des ressources via HTTP (GET, POST, PUT, DELETE). TrackR expose une API REST consommée par le frontend React.
Argon2id	Algorithme de hashage de mots de passe (vainqueur Password Hashing Competition 2015). Combine résistance aux attaques GPU (Argon2d) et aux canaux auxiliaires (Argon2i). Recommandé OWASP 2023. Utilisé par défaut dans Symfony pour les comptes freelances.
CDA	Concepteur Développeur d'Applications. Titre RNCP37873 niveau 6 (Bac+3) délivré par La Plateforme, Marseille.
CGI (art. 289)	Code Général des Impôts, article 289 : liste les 16 mentions obligatoires sur une facture en France.
CI/CD	Continuous Integration / Continuous Deployment. Pipeline automatisé : tests, build, déploiement à chaque push sur main.
Docker multi-stage	Technique Dockerfile séparant build (avec dépendances de dev) et runtime (image minimale). Réduit la taille de l'image de production.
DomPDF	Librairie PHP générant des PDF à partir de HTML/CSS. Utilisée pour les factures et devis TrackR.
JWT	JSON Web Token. Jeton signé cryptographiquement utilisé pour l'authentification stateless du freelance.
Loi anti-fraude 2018	Loi 2016-1918 imposant l'immutabilité des factures : une facture émise ne peut plus être modifiée ni supprimée.
Magic-link	Lien d'accès unique envoyé par email. Le client portail s'authentifie sans mot de passe via ce lien (token 64 chars, TTL 7j, usage unique).
MCD	Modèle Conceptuel de Données (méthode Merise). Représentation abstraite des entités et de leurs relations avant traduction en SQL.
Mercure	Protocole basé sur SSE (Server-Sent Events) et HTTP/2 pour les notifications temps réel. Supporté nativement par Symfony.



Multi-tenant	Architecture où une même instance de l'application sert plusieurs clients (ici : plusieurs freelances) avec isolation stricte des données.
RNCP	Repertoire National des Certifications Professionnelles. Le titre CDA est référencé RNCP37873.
SSE (Server-Sent Events)	Protocole HTTP unidirectionnel (serveur vers client) utilisé par Mercure pour les notifications temps réel. Plus simple que WebSockets pour les flux en lecture seule.
ShadCN/UI	Librairie de composants React (base Radix UI + Tailwind). Les composants sont copiés dans le projet, pas encapsulés dans une dépendance.
Snapshot client	Champ JSONB sur la table <code>facture</code> contenant les coordonnées du client figées au moment de l'émission. Exigence légale.
Stripe Connect	Programme Stripe permettant à une plateforme de gérer les paiements pour le compte de tiers (ici : le freelance). Mode Standard retenu.
Symfony Messenger	Composant Symfony pour le traitement asynchrone de messages (emails, PDF). La file est stockée dans Redis, le consumer tourne en worker Docker.
TVA art. 293B	Franchise de TVA applicable aux micro-entreprises sous un certain seuil. La mention "TVA non applicable : art. 293 B du CGI" remplace les lignes de TVA sur la facture.
White-label	Produit dont l'interface est entièrement rebrandable au nom d'un tiers. Le portail client TrackR n'affiche jamais le nom "TrackR".
Backed Enum (PHP)	Enum PHP 8.1 dont chaque cas porte une valeur scalaire (string ou int). Exemple : <code>StatutFacture::Payee→value === 'payee'</code> . Doctrine ORM 3 mappe automatiquement les backed enums sur des colonnes SQL.
Traefik	Reverse proxy et load balancer écrit en Go. Détecte automatiquement les conteneurs Docker via les labels et gère les certificats SSL via Let's Encrypt (ACME). Utilisé en production TrackR (v3.3) pour router le trafic vers les conteneurs <code>frontend</code> et <code>mercure</code> .
Docker Compose	Outil d'orchestration de conteneurs Docker. Définit les services, réseaux et volumes dans <code>docker-compose.yml</code> . Utilisé en développement local (8 services) et en production (7 services).
DQL	Doctrine Query Language. SQL orienté objet propre à Doctrine ORM. Permet de requêter des entités PHP plutôt que des tables SQL directement.
eIDAS SES	Simple Electronic Signature au sens du règlement européen eIDAS. Signature électronique de base (HTML canvas + métadonnées IP/timestamp). Niveau retenu pour la signature de devis dans TrackR.



HMAC	Hash-based Message Authentication Code. Fonction de verification d'integrite utilisant un secret partage. TrackR l'utilise pour securiser le parametre <code>state</code> du flux OAuth Stripe (<code>hash_hmac('sha256', ...)</code>)
httpOnly (cookie)	Attribut de cookie rendant la valeur inaccessible depuis JavaScript. Le refresh token JWT et le <code>session_token</code> portail sont stockes en cookies httpOnly pour prevenir le vol via XSS.
LME	Loi de Modernisation de l'Economie (2008). Impose les penalites de retard de paiement et l'indemnite forfaitaire de recouvrement de 40 EUR mentionnees sur les factures B2B.
MoSCoW	Methode de priorisation : Must have, Should have, Could have, Won't have. Appliquee aux 25 User Stories de TrackR pour distinguer le MVP des fonctionnalites futures.
PHPUnit	Framework de tests pour PHP. TrackR l'utilise en mode WebTestCase pour envoyer de vraies requetes HTTP au kernel Symfony et verifier les reponses avec une base PostgreSQL reelle.
Redis	Base de donnees en memoire (key-value store). Utilise par Symfony Messenger comme file d'attente de messages (emails async, PDF). Persistence AOF activee en production (append-only file).
RSA-256	Algorithme de signature asymetrique. LexikJWTAuthenticationBundle genere les tokens JWT avec une paire de cles RSA 2048 bits. La cle privee signe, la cle publique verifie.
Snapshot	Copie figee de donnees au moment d'un evenement. En facturation, le <code>snapshot_client</code> contient les coordonnees exactes du client au moment de l'emission : il n'est jamais mis a jour.
Supervisor	Gestionnaire de processus Unix. Lance et surveille le worker Symfony Messenger en production. Redemarrage automatique si le process s'arrete (exit code non nul ou timeout).
Vitest	Framework de tests unitaires JavaScript compatible Vite. Utilise pour tester les hooks et composants React du dashboard TrackR. S'execute avec <code>npm run test : --run</code> en CI.